



4장.분류 – Part 2

4팀 – 최혜윤, 김태영, 정윤지

목차

#01 4.5장 ~ 4.6장

#02 4.7장 ~ 4.8장

#03 4.10장 ~ 4.11장



4.5 GBM



4.5 GBM(Gradient Boosting Machine)

부스팅 알고리즘

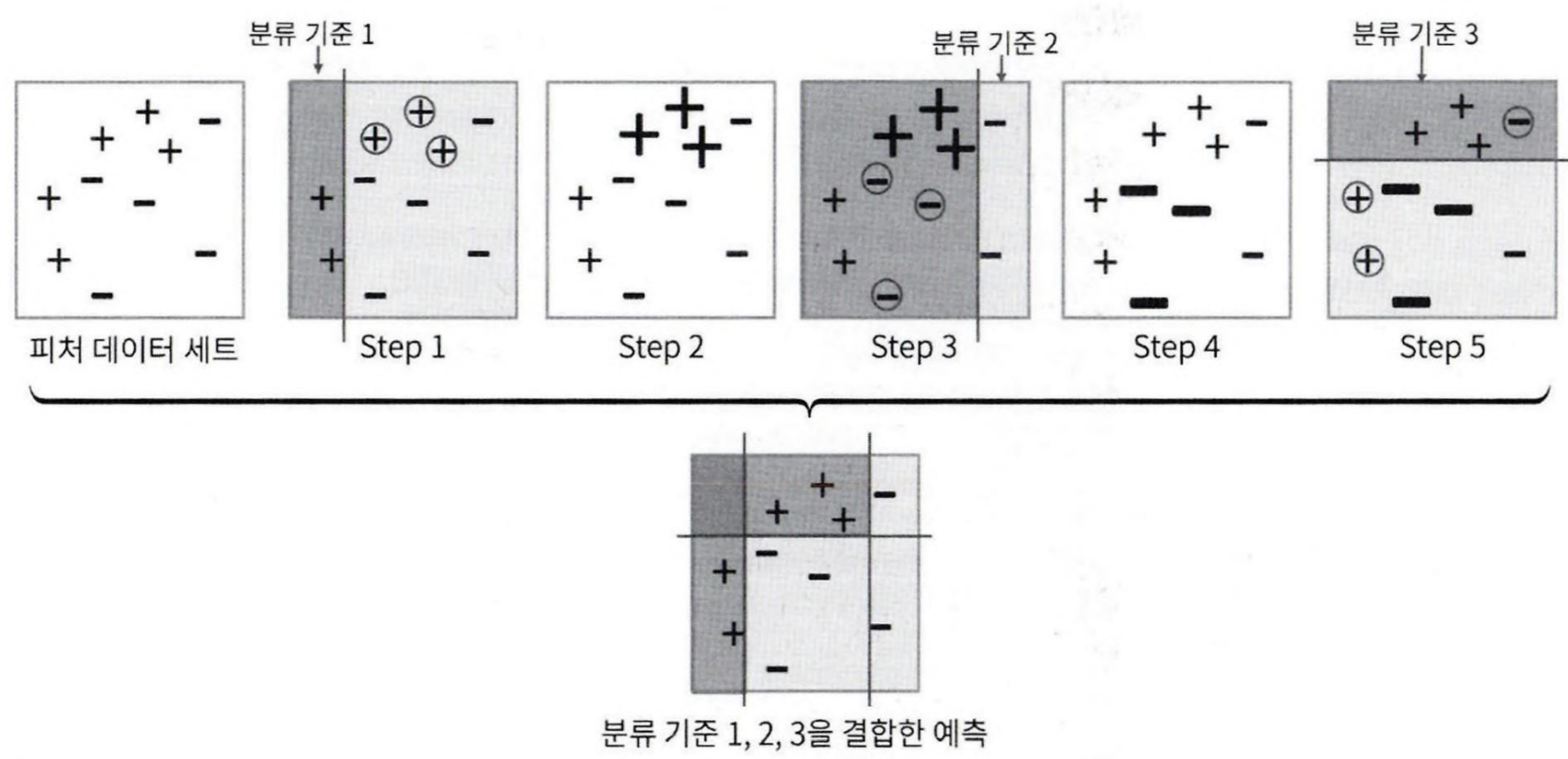
여러 개의 약한 학습기(weak learner)를 순차적으로 학습-예측
→ 잘못 예측한 데이터에 가중치 부여 → 오류를 개선하는 학습 방식.

〈대표적인 부스팅 구현〉

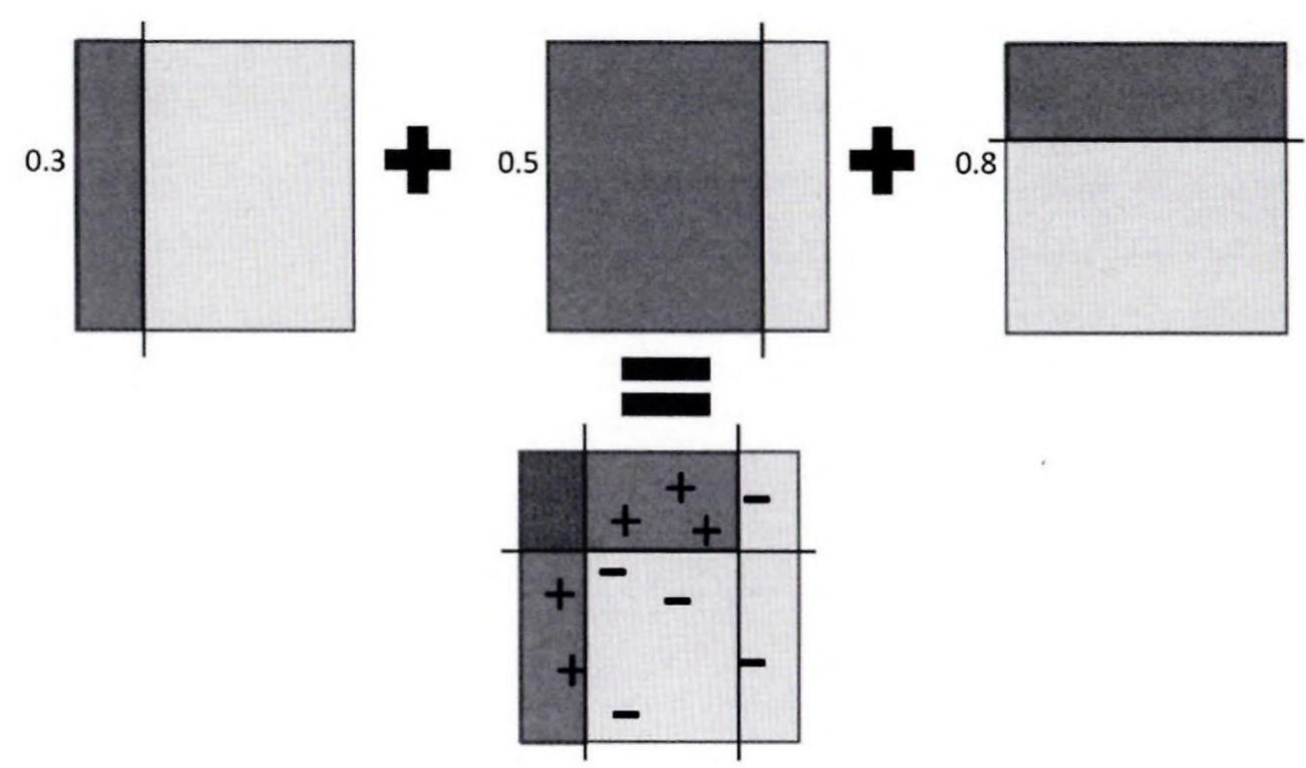
1. AdaBoost(Adaptive boosting)
2. Gradient Boost

4.5 GBM(Gradient Boosting Machine)

1. AdaBoost(Adaptive boosting)



AdaBoost 학습 방식



개별 약한 학습기

4.5 GBM(Gradient Boosting Machine)

2. Gradient Boost Machine(GBM)

AdaBoost와 유사하나, 가중치 업데이트를 경사 하강법(Gradient Descent)을 이용하는 것이 큰 차이.

- 분류의 실제 결괏값 : y
- 피처 : $x_1, x_2, \dots, x_n$
- 예측 함수 : $F(x)$
- 오류식 : $h(x) = y - F(x)$

Gradient Descent = 오류식을 최소화하는 방향성 $\rightarrow$ 반복적으로 가중치 값 업데이트 !

4.5 GBM(Gradient Boosting Machine)

2. Gradient Boost Machine(GBM)

GradientBoostingClassifier - 사용자 행동 데이터 세트 예측 분류

```
[56] from sklearn.ensemble import GradientBoostingClassifier
import time
import warnings
warnings.filterwarnings('ignore')

X_train, X_test, y_train, y_test = get_human_dataset()

# GBM 수행 시간 측정을 위함. 시작 시간 설정.
start_time = time.time()
gb_clf = GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train, y_train)
gb_pred = gb_clf.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)

print('GBM 정확도: {0:.4f}'.format(gb_accuracy))
print("GBM 수행 시간: {0:.1f} 초 ".format(time.time() - start_time))
```

GBM 정확도: 0.9379
GBM 수행 시간: 1016.3 초

- 예측 성능 : GBM > 랜덤 포레스트

- But, 긴 수행시간**
(약한 학습기의 순차적인 예측 오류 보정
→ 병렬 지원 X)

& 하이퍼 파라미터 튜닝 필요

4.5 GBM(Gradient Boosting Machine)

GBM 하이퍼 파라미터 소개

- **loss** : 경사 하강법에서 사용할 비용 함수를 지정.
- **learning_rate** : GBM에서 Weak learner가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수.
- **n_estimators** : weak learner의 개수.
- **subsample** : weak learner가 학습에 사용하는 데이터 샘플링 비율.

4.5 GBM(Gradient Boosting Machine)

GBM 하이퍼 파라미터 튜닝

```
[122] from sklearn.model_selection import GridSearchCV

params = {
    'n_estimators' : [100, 500],
    'learning_rate' : [0.05, 0.1]
}
grid_cv = GridSearchCV(gb_clf, param_grid=params, cv=2, verbose=1)
grid_cv.fit(X_train, y_train)
print('최적 하이퍼 파라미터:Wn', grid_cv.best_params_)
print('최고 예측 정확도: {0:.4f}'.format(grid_cv.best_score_))

# GridSearchCV를 이용해 최적으로 학습된 estimator로 예측 수행.
gb_pred = grid_cv.best_estimator_.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)
print("GBM 정확도: {0:.4f}".format(gb_accuracy))
```



Fitting 2 folds for each of 4 candidates, totalling 8 fits

최적 하이퍼 파라미터:

`{'learning_rate': 0.1, 'n_estimators': 500}`

최고 예측 정확도: 0.9517

GBM 정확도: 0.9737

- 학습 데이터 → 모든 하이퍼 파라미터 조합에 대해 교차 검증 실행.
- 각 조합마다 평균 성능 계산.
- 가장 성능 좋은 조합을 `best_params_`로 지정.
- 최적 모델로 테스트 데이터 예측.
- 최종 정확도 출력.

4.6 XGBoost



4.6 XGBoost(eXtra Gradient Boost)

XGBoost 개요

- 트리 기반 앙상블 학습에서 가장 각광받고 있는 알고리즘 중 하나.
- GBM에 기반 → 느린 수행 시간 & 과적합 규제(Regularization) 부재 등의 문제 해결
- 병렬 CPU 환경에서 병렬 학습 0 → 빠른 학습 가능화
- 핵심 라이브러리 : C / C++
- xgboost 패키지 = 파이썬 wrapper XGBoost 모듈 + 사이킷런 wrapper XGBoost 모듈

4.6 XGBoost(eXtra Gradient Boost)

XGBoost 장점

뛰어난 예측 성능	일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘합니다.
GBM 대비 빠른 수행 시간	일반적인 GBM은 순차적으로 Weak learner가 가중치를 증감하는 방법으로 학습하기 때문에 전반적으로 속도가 느립니다. 하지만 XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장합니다. 아쉽게도 XGBoost가 일반적인 GBM에 비해 수행 시간이 빠르다는 것이지, 다른 머신러닝 알고리즘(예를 들어 랜덤 포레스트)에 비해서 빠르다는 의미는 아닙니다.
과적합 규제 (Regularization)	표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 좀 더 강한 내구성을 가질 수 있습니다.
Tree pruning (나무 가지치기)	일반적으로 GBM은 분할 시 부정 손실이 발생하면 분할을 더 이상 수행하지 않지만, 이러한 방식도 자칫 지나치게 많은 분할을 발생할 수 있습니다. 다른 GBM과 마찬가지로 XGBoost도 max_depth 파라미터로 분할 깊이를 조정하기도 하지만, tree pruning으로 더 이상 긍정 이득이 없는 분할을 가지치기 해서 분할 수를 더 줄이는 추가적인 장점을 가지고 있습니다.
자체 내장된 교차 검증	XGBoost는 반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증을 수행해 최적화된 반복 수행 횟수를 가질 수 있습니다. 지정된 반복 횟수가 아니라 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복을 중간에 멈출 수 있는 조기 중단 기능이 있습니다.
결손값 자체 처리	XGBoost는 결손값을 자체 처리할 수 있는 기능을 가지고 있습니다.

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 하이퍼 파라미터

- ✓ 파이썬 wrapper XGBoost 모듈 & 사이킷런 wrapper XGBoost 모듈
 - 일부 하이퍼 파라미터가 다름 (동일한 기능을 하지만, 파라미터 명이 다름)
- 1. 일반 파라미터: 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터
- 2. 부스터 파라미터: 트리 최적화, 부스팅, regularization 등과 관련된 파라미터
- 3. 학습 태스크 파라미터: 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 하이퍼 파라미터

1. 주요 일반 파라미터

- `booster`: `gbtree`(tree based model) 또는 `gblinear`(linear model) 선택. 디폴트는 `gbtree`입니다.
- `silent`: 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- `nthread`: CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 하이퍼 파라미터

2. 부스터 파라미터

- `eta [default=0.3, alias: learning_rate]`: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 `xgboost`를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 `eta`는 `learning_rate` 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- `num_boost_rounds`: GBM의 `n_estimators`와 같은 파라미터입니다.
- `min_child_weight [default=1]`: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 `weight` 총합. `min_child_weight`이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- `gamma [default=0, alias: min_split_loss]`: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- `max_depth [default=6]`: 트리 기반 알고리즘의 `max_depth`와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. `Max_depth`가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- `sub_sample [default=1]`: GBM의 `subsample`과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. `sub_sample=0.5`로 지정하면 전체 데이터의 절반을 트리를 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- `colsample_bytree [default=1]`: GBM의 `max_features`와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- `lambda [default=1, alias: reg_lambda]`: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `alpha [default=0, alias: reg_alpha]`: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `scale_pos_weight [default=1]`: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터입니다.

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 하이퍼 파라미터

3. 학습 태스크 파라미터

- **objective**: 최솟값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- **binary:logistic**: 이진 분류일 때 적용합니다.
- **multi:softmax**: 다중 분류일 때 적용합니다. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num_class 파라미터를 지정해야 합니다.
- **multi:softprob**: multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환합니다.
- **eval_metric**: 검증에 사용되는 함수를 정의합니다. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error입니다. 다음은 eval_metric의 값 유형입니다.
 - **rmse**: Root Mean Square Error
 - **mae**: Mean Absolute Error
 - **logloss**: Negative log-likelihood
 - **error**: Binary classification error rate (0.5 threshold)
 - **merror**: Multiclass classification error rate
 - **mlogloss**: Multiclass logloss
 - **auc**: Area under the curve

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 하이퍼 파라미터

<Overfitting 문제 해결 방법>

- eta 값을 낮추기.(0.01 ~ 0.1) → num_round(or n_estimators)는 높이기.
- max_depth 값 낮추기.
- min_child_weight 값 높이기.
- gamma 값 높이기.
- subsample, colsample_bytree 조정하기.

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- xgboost – 자체적으로 교차 검증, 성능 평가, 피쳐 중요도 등의 시각화(plotting) 기능.
- 조기 중단 - 더 이상 예측 오류가 개선 X → 반복 중지 → 수행 시간 개선.
- 위스콘신 유방암 데이터 세트 - 종양 크기, 모양 등의 속성값 기반으로 악성 종양(malignant)인지, 양성 종양(benign)인지 분류.

양성 종양 – 성장 속도 느림, 전이 X.

악성 종양 – 빠르게 성장, 신체 부위에 확산, 전이.

- 다양한 피처에 따라 악성 종양(malignant)인지, 양성 종양(benign)인지 XGBoost 이용해 예측.

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

1) 파이썬 패키지에서 xgboost와 필요한 모듈 임포트, 데이터 확인

```
[ ] import xgboost as xgb
    from xgboost import plot_importance
    import pandas as pd
    import numpy as np
    from sklearn.datasets import load_breast_cancer
    from sklearn.model_selection import train_test_split
    import warnings
    warnings.filterwarnings('ignore')

    dataset = load_breast_cancer()
    features = dataset.data
    labels = dataset.target

    cancer_df = pd.DataFrame(data=features, columns = dataset.feature_names)
    cancer_df['target'] = labels
    cancer_df.head(3)
```



	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430

3 rows × 31 columns

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

2) 레이블 값 분포 확인.('malignant(악성)' = 0 / 'benign(양성)' = 1)

```
▶ print(dataset.target_names)  
print(cancer_df['target'].value_counts())
```

```
⇒ ['malignant' 'benign']  
target  
1      357  
0      212  
Name: count, dtype: int64
```

- 양성(benign) 1 : 357개
- 악성(malignant) 0 : 212개

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

3) train data, test data 분할하기

```
[ ] # cancer_df에서 feature용 DataFrame과 Label용 Series 객체 추출
    # 맨 마지막 칼럼이 Label임. Feature용 DataFrame은 cancer_df의 첫번째 칼럼에서 맨 마지막 두번째 칼럼까지를 :-1 슬라이싱으로 추출.
    X_features = cancer_df.iloc[:, :-1]
    y_label = cancer_df.iloc[:, -1]

    # 전체 데이터 중 80%는 학습용 데이터, 20%는 테스트용 데이터 추출
    X_train, X_test, y_train, y_test = train_test_split(X_features, y_label, test_size=0.2, random_state=156)

    # 위에서 만든 X_train, y_train을 다시 쪼개서 90%는 학습과 10%는 검증용 데이터로 분리
    X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=156)

    print(X_train.shape, X_test.shape)
    print(X_tr.shape, X_val.shape)
```

```
⇒ (455, 30) (114, 30)
   (409, 30) (46, 30)
```


4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

4) Dmatrix 객체 생성 & 하이퍼 파라미터 설정

```
[ ] # 만약 구버전 XGBoost에서 DataFrame으로 DMatrix 생성이 안 될 경우 X_train.values로 넘파이 변환.  
# 학습, 검증, 테스트용 DMatrix를 생성.  
dtr = xgb.DMatrix(data=X_tr, label=y_tr)  
dval = xgb.DMatrix(data=X_val, label=y_val)  
dtest = xgb.DMatrix(data=X_test, label=y_test)
```

<파이썬 VS 사이킷런 wrapper XGBoost>

- 파이썬 : XGBoost만의 전용 데이터 객체인 Dmatrix 사용.
- Dmatrix 주요 입력 파라미터 :
data(피쳐 데이터 세트)
& label(분류 - 레이블 데이터 세트 /
회귀 - 종속값 데이터 세트)

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

4) Dmatrix 객체 생성 & 하이퍼 파라미터 설정

```
[ ] params = {'max_depth' : 3,  
              'eta' : 0.05,  
              'objective' : 'binary:logistic',  
              'eval_metric' : 'logloss'  
            }  
num_rounds = 400
```

〈XGBoost 하이퍼 파라미터 설정〉

- 딕셔너리 형태
- max_depth : 3
- eta : 0.1
- 목적 함수 - 이진 로지스틱
- 평가 성능 지표 - logloss
- num_rounds : 400회

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

5) 학습 & 결과 확인

```
# 학습 데이터 셋은 'train' 또는 평가 데이터 셋은 'eval' 로 명기합니다.
eval_list = [(dtr,'train'),(dval,'eval')] # 또는 eval_list = [(dval,'eval')] 만 명기해도 무방.

# 하이퍼 파라미터와 early stopping 파라미터를 train( ) 함수의 파라미터로 전달
xgb_model = xgb.train(params = params , dtrain=dtr , num_boost_round=num_rounds , \
                      early_stopping_rounds=50, evals=eval_list )
```

【Output】

```
[0] train-logloss:0.65016 eval-logloss:0.66183
[1] train-logloss:0.61131 eval-logloss:0.63609
[2] train-logloss:0.57563 eval-logloss:0.61144
.....
[125] train-logloss:0.01998 eval-logloss:0.25714
[126] train-logloss:0.01973 eval-logloss:0.25587 ★
[127] train-logloss:0.01946 eval-logloss:0.25640
[128] train-logloss:0.01927 eval-logloss:0.25685
.....
[175] train-logloss:0.01267 eval-logloss:0.26086
[176] train-logloss:0.01258 eval-logloss:0.26103
```

- 하이퍼 파라미터를 xgboost모듈의 train() 함수에 파라미터로 전달.
- 성능 평가 – 별도의 검증 데이터 세트
- 반드시 **평가용 데이터 세트** 지정, eval_metric 설정 !
- **평가용 데이터 세트:**
[(dtr, 'train'), (dval, 'eval')]
 - dtr : 학습용 Dmatrix
 - dval : 검증용 Dmatrix
- Xgboost 학습 시 → 평가 지표 결과 출력

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

5) 학습 & 결과 확인

```
0초
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score

def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}, F1: {3:.4f}, AUC:{4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))

get_clf_eval(y_test, preds, pred_probs)
```

오차 행렬
[[35 2]
 [2 75]]
정확도: 0.9649, 정밀도: 0.9740, 재현율: 0.9740, F1: 0.9740, AUC:0.9965

```
[ ] pred_probs = xgb_model.predict(dtest)
    print('predict() 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨')
    print(np.round(pred_probs[:10], 3))

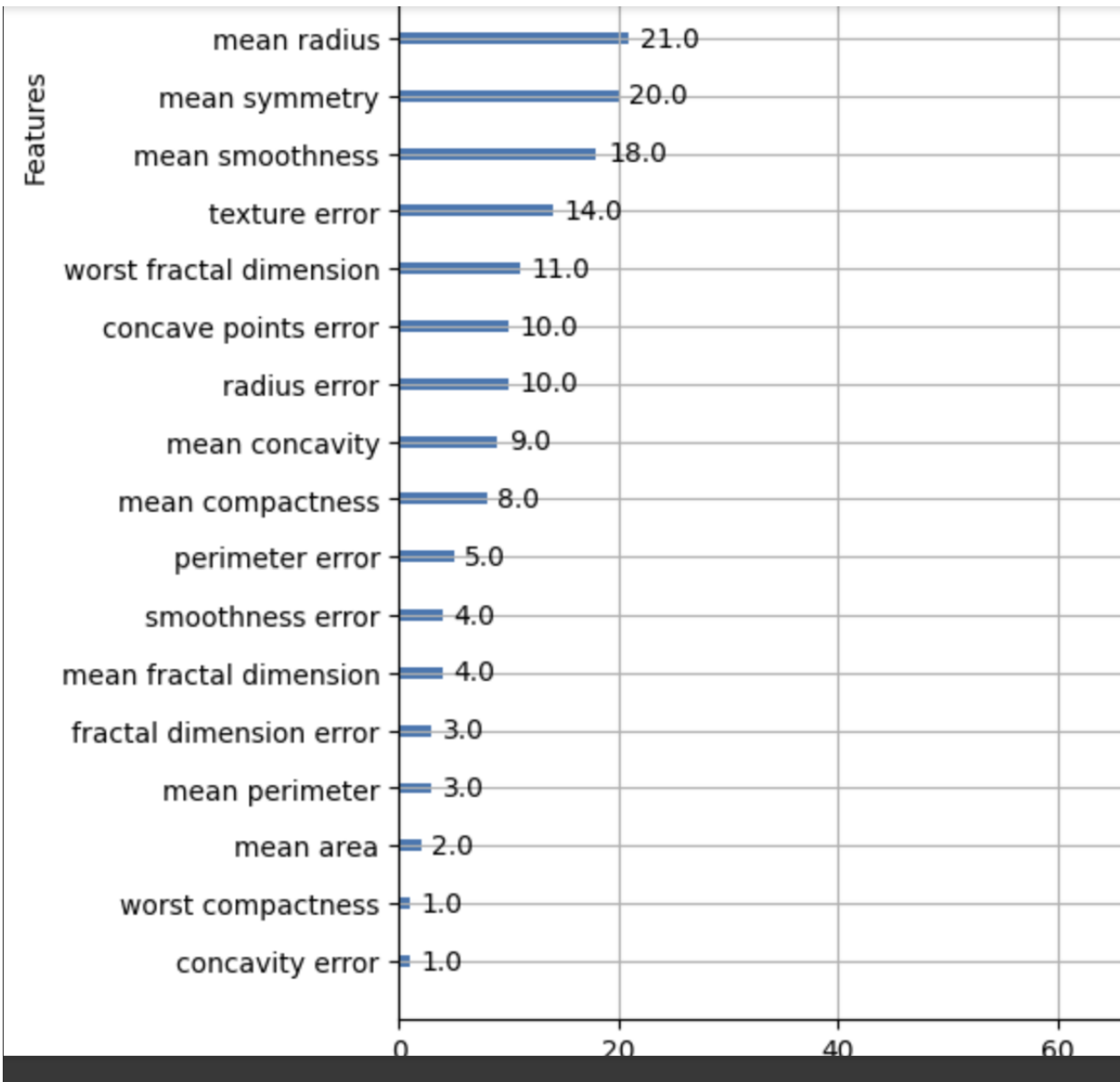
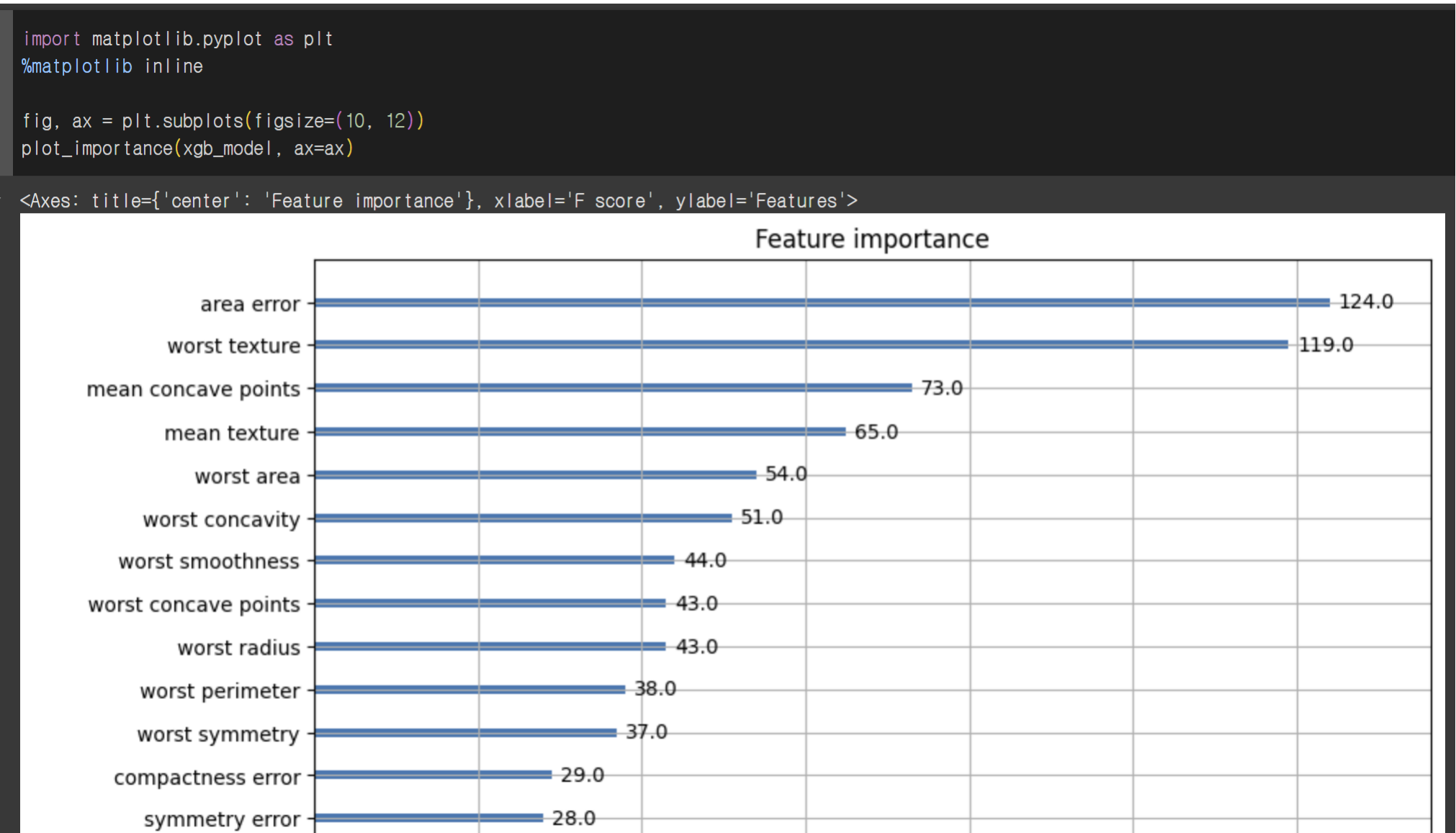
    # 예측 확률이 0.5보다 크면 1, 그렇지 않으면 0으로 예측값 결정하여 List 객체인 preds에 저장
    preds = [1 if x > 0.5 else 0 for x in pred_probs]
    print('예측값 10개만 표시:', preds[:10])
```

predict() 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨
[0.938 0.004 0.75 0.049 0.98 1. 0.999 0.999 0.998 0.001]
예측값 10개만 표시: [1, 0, 1, 0, 1, 1, 1, 1, 1, 0]

4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

6) 시각화



4.6 XGBoost(eXtra Gradient Boost)

파이썬 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- cv() API & 파라미터

```
xgboost.cv(params, dtrain, num_boost_round=10, nfold=3, stratified=False, folds=None, metrics=(),  
obj=None, feval=None, maximize=False, early_stopping_rounds=None, fpreproc=None, as_pandas=True,  
verbose_eval=None, show_stdv=True, seed=0, callbacks=None, shuffle=True)
```

- params (dict): 부스터 파라미터
- dtrain (DMatrix): 학습 데이터
- num_boost_round (int): 부스팅 반복 횟수
- nfold (int): CV 폴드 개수.
- stratified (bool): CV 수행 시 층화 표본 추출(stratified sampling) 수행 여부
- metrics (string or list of strings): CV 수행 시 모니터링할 성능 평가 지표
- early_stopping_rounds (int): 조기 중단을 활성화시킴. 반복 횟수 지정.

- xgb.cv 반환값 :
DataFrame 형태

4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost의 개요

- 사이킷런의 기본 Estimator 기반으로 생성 → 다른 Estimator와 동일하게 fit()과 predict()만으로 학습, 예측 가능 .
- GridSearchCV, Pipeline 등의 유틸리티 그대로 사용 .
- 분류를 위한 래퍼 클래스 : XGBClassifier
- 회귀를 위한 래퍼 클래스 : XGBRegressor

〈사이킷런 래퍼 XGBoost 하이퍼 파라미터 변경〉

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- XGBClassifier import, 학습 & 예측 수행

```
# 사이킷런 래퍼 XGBoost 클래스인 XGBClassifier 임포트
from xgboost import XGBClassifier

# Warning 메시지를 없애기 위해 eval_metric 값을 XGBClassifier 생성 인자로 입력.
xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3,
                             eval_metric='logloss')
xgb_wrapper.fit(X_train, y_train, verbose=True)
w_preds = xgb_wrapper.predict(X_test)
w_pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]
```

```
get_clf_eval(y_test , w_preds, w_pred_proba)
```

오차 행렬

[[34 3]

[1 76]]

정확도: 0.9649, 정밀도: 0.9620, 재현율: 0.9870, F1: 0.9744, AUC:0.9954

- 파이썬 wrapper XGBoost보다 좋은 평가 결과!

- 파이썬 : 조기 종단을 위한 최초 학습 데이터인 X_train을 다시 X_tr , X_val로 분리 → 최종학습 데이터 건수가 감소.

- 위스콘신 데이터 세트 : 작음

→ 검증 데이터 분리 or 교차 검증 → 성능 수치 불안정

- 데이터 건수 많은 경우

→ 원본 학습 데이터를 학습과 검증 데이터로 분리, 조기 중단 횟수 적절하게 부여 → 과적합 개선 → 모델 성능 향상

4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- 조기 중단 수행

```
from xgboost import XGBClassifier

xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=3)
evals = [(X_tr, y_tr), (X_val, y_val)]
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric="logloss",
                eval_set=evals, verbose=True)

ws50_preds = xgb_wrapper.predict(X_test)
ws50_pred_proba = xgb_wrapper.predict_proba(X_test)[:, 1]
```

[Output]

```
[0] validation_0-logloss:0.65016    validation_1-logloss:0.66183
[1] validation_0-logloss:0.61131    validation_1-logloss:0.63609
[2] validation_0-logloss:0.57563    validation_1-logloss:0.61144
.....
[125] validation_0-logloss:0.01998    validation_1-logloss:0.25714
[126] validation_0-logloss:0.01973    validation_1-logloss:0.25587
[127] validation_0-logloss:0.01946    validation_1-logloss:0.25640
.....
[175] validation_0-logloss:0.01267    validation_1-logloss:0.26086
[176] validation_0-logloss:0.01258    validation_1-logloss:0.26103
```

- 조기 중단 관련 파라미터를 fit()에 입력.
- early_stopping_rounds : 반복 횟수 정의
- eval_metric : 조기 종단을 위한 평가 지표
- eval_set : 성능 평가 수행 데이터 세트

- 최초 학습 데이터인 X_train을 다시 X_tr ,
X_val로 분리 → 학습 & 조기 중단 적용

- [(X_tr, y_tr), (X_val, y_val)] : 앞 튜플 - 학습용
뒤 튜플 - 검증용

4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- 조기 중단 학습한 XGBClassifier 예측 성능 확인

```
get_clf_eval(y_test , ws50_preds, ws50_pred_proba)
```

【Output】

오차 행렬

[[34 3]

[2 75]]

정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC:0.9933

- 파이썬 wrapper XGBoost와 동일!
- 위스콘신 데이터 세트 : 작음
→ 검증 데이터 분리 or 교차 검증
→ 성능 수치 불안정

4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- early_stopping_rounds=10으로 설정, 예측 재수행

```
# early_stopping_rounds를 10으로 설정하고 재학습.
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=10,
                eval_metric="logloss", eval_set=evals, verbose=True)

ws10_preds = xgb_wrapper.predict(X_test)
ws10_pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]
get_clf_eval(y_test , ws10_preds, ws10_pred_proba)
```

【Output】

```
[0] validation_0-logloss:0.65016    validation_1-logloss:0.66183
[1] validation_0-logloss:0.61131    validation_1-logloss:0.63609
[2] validation_0-logloss:0.57563    validation_1-logloss:0.61144
...
[92] validation_0-logloss:0.03152    validation_1-logloss:0.25918
[93] validation_0-logloss:0.03107    validation_1-logloss:0.25865
[94] validation_0-logloss:0.03049    validation_1-logloss:0.25951
...
```

```
[101] validation_0-logloss:0.02751    validation_1-logloss:0.25955
[102] validation_0-logloss:0.02714    validation_1-logloss:0.25901
[103] validation_0-logloss:0.02668    validation_1-logloss:0.25991
```

오차 행렬

```
[[34  3]
```

```
[ 3 74]]
```

정확도: 0.9474, 정밀도: 0.9610, 재현율: 0.9610, F1: 0.9610, AUC:0.9933

- Early_stopping_rounds = 50
일 때 보다 정확도 저하!

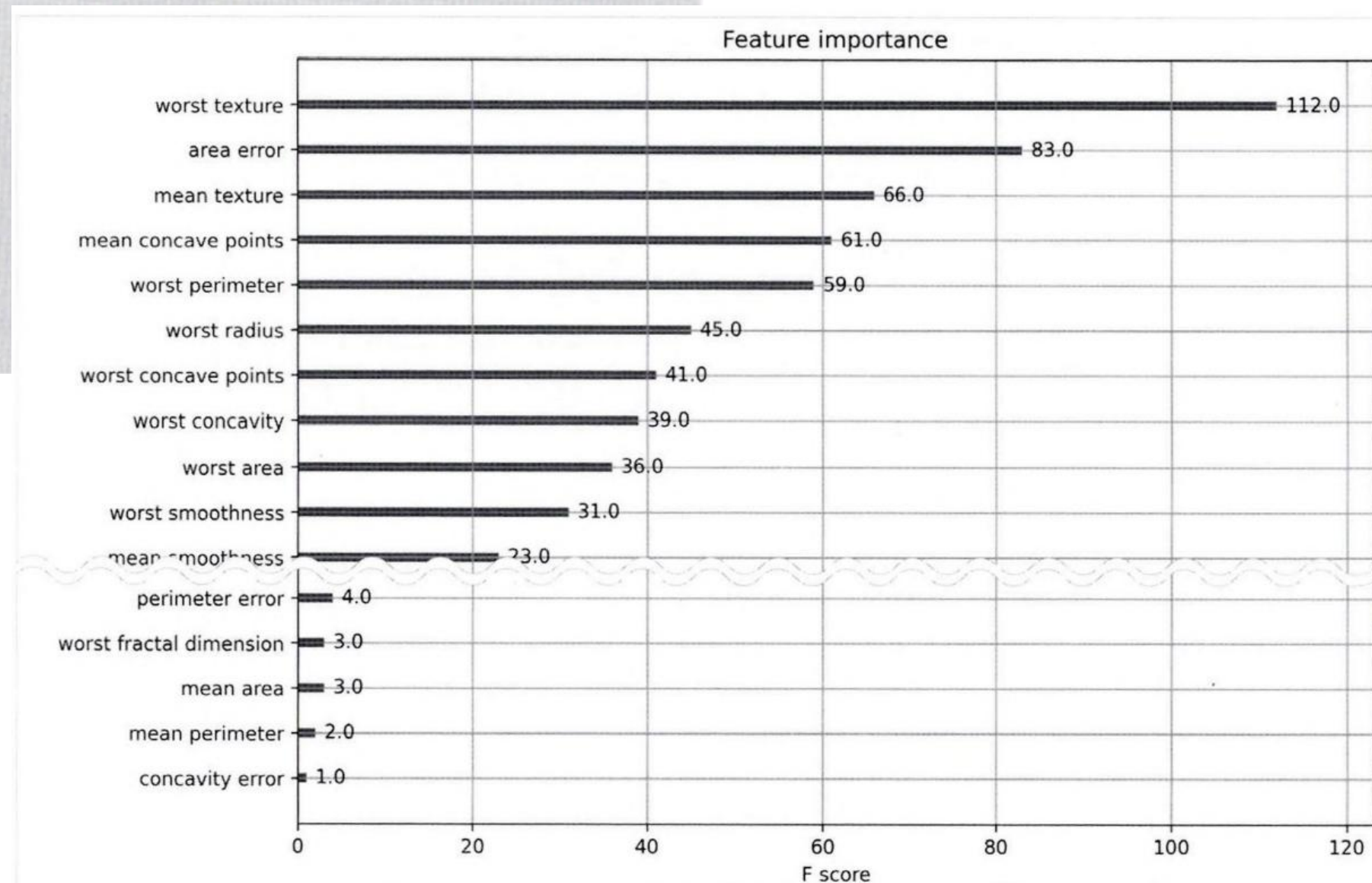
4.6 XGBoost(eXtra Gradient Boost)

사이킷런 wrapper XGBoost 적용 – 위스콘신 유방암 예측

- 시각화

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
# 사이킷런 래퍼 클래스를 입력해도 무방.
plot_importance(xgb_wrapper, ax=ax)
```



4.7 LightBGM



LightBGM

주요 일반 파라미터

- `boost`: `gbtree`(tree based model) 또는 `gblinear`(linear model) 선택. 디폴트는 `gbtree`입니다.
- `silent`: 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- `nthread`: CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

주요 부스터 파라미터

- `eta` [default=0.3, alias: `learning_rate`]: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파인션 래퍼 기반의 `xgboost`를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 `eta`는 `learning_rate` 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- `num_boost_rounds`: GBM의 `n_estimators`와 같은 파라미터입니다.
- `min_child_weight`[default=1]: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 `weight` 총합. `min_child_weight`이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- `gamma` [default=0, alias: `min_split_loss`]: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- `max_depth`[default=6]: 트리 기반 알고리즘의 `max_depth`와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. `Max_depth`가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- `sub_sample`[default=1]: GBM의 `subsample`과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. `sub_sample=0.5`로 지정하면 전체 데이터의 절반을 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- `colsample_bytree`[default=1]: GBM의 `max_features`와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- `lambda` [default=1, alias: `reg_lambda`]: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `alpha` [default=0, alias: `reg_alpha`]: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.

학습 태스크 파라미터

- `objective`: 최소값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- `binary:logistic`: 이진 분류일 때 적용합니다.
- `multi:softmax`: 다중 분류일 때 적용합니다. 손실함수가 `multi:softmax`일 경우에는 레이블 클래스의 개수인 `num_class` 파라미터를 지정해야 합니다.
- `multi:softprob`: `multi:softmax`와 유사하나 개별 레이블 클래스의 해당되는 예측 확률들을 반환합니다.
- `eval_metric`: 검증에 사용되는 함수를 정의합니다. 기본값은 `rmse`인 경우는 `rmse`, 분류일 경우에는 `error`입니다. 다음은 `eval_metric`의 값 유형입니다.
 - `rmse`: Root Mean Square Error
 - `mae`: Mean Absolute Error
 - `logloss`: Negative log-likelihood
 - `error`: Binary classification error rate (0.5 threshold)
 - `merror`: Multiclass classification error rate
 - `mlogloss`: Multiclass logloss
 - `auc`: Area under the curve

➔ GridSearchCV로 하이퍼 파라미터 튜닝
=> 수행 시간   

=>>> LightBGM

장점

1. 더 빠른 학습과 예측 수행 시간
2. 더 작은 메모리 사용량
3. 카테고리형 피쳐의 자동 변환 & 최적 분할

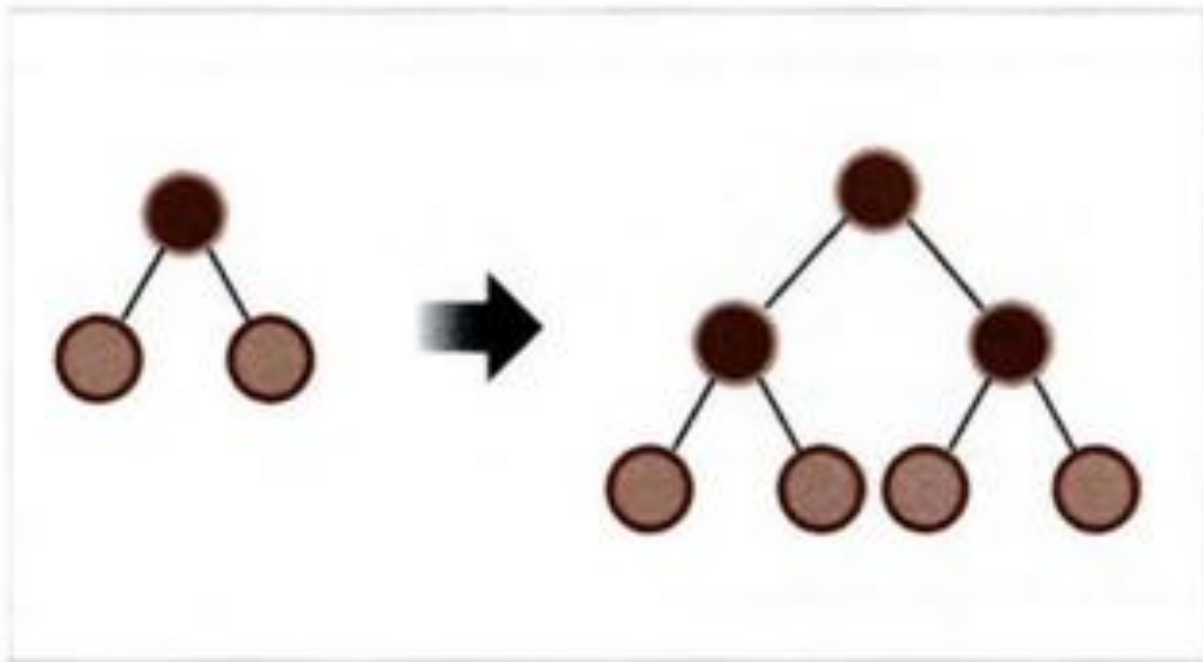
단점

10,000건 이하의 데이터 세트에 적용하면 과적합이 발생하기 쉬움

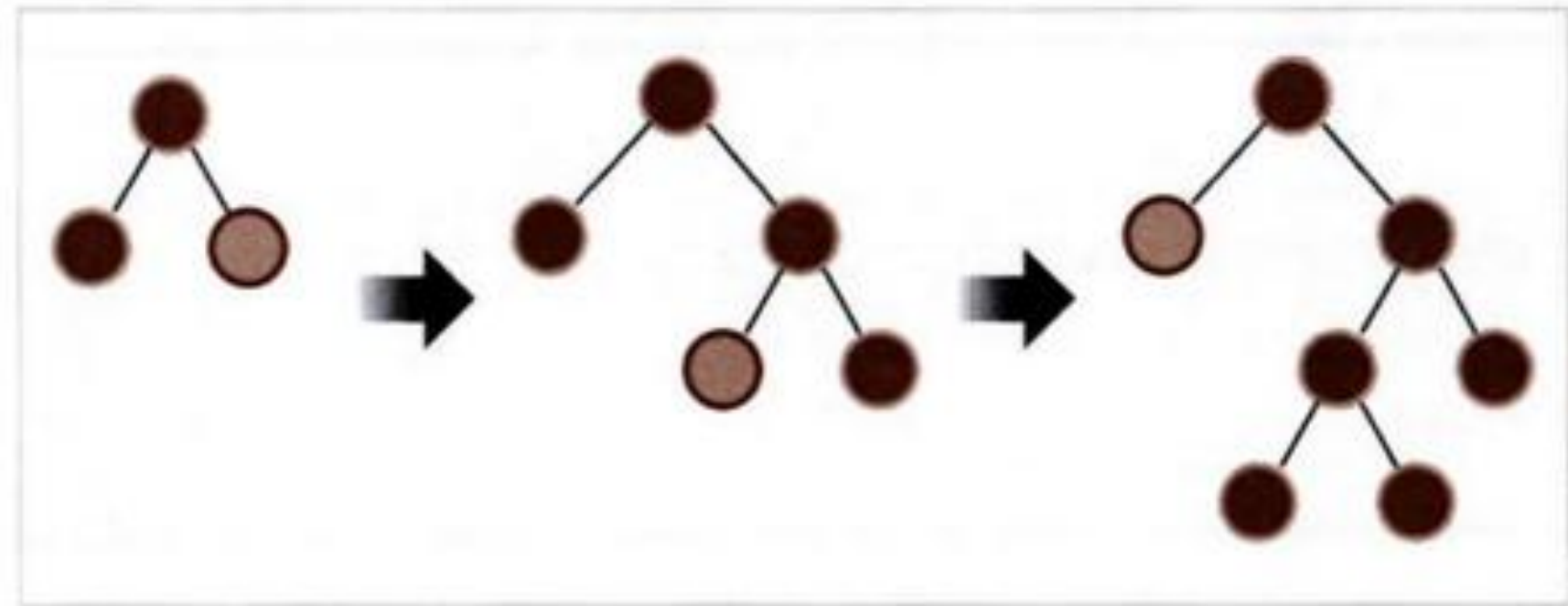
LightBGM

리프 중심 트리 분할(Leaf Wise) 방식 사용

균형 트리 분할(Level Wise)



리프 중심 트리 분할(Leaf Wise)



- 최대한 균형 잡힌 트리 유지하면서 분할
- 트리의 깊이 최소화
- 오버피팅에 👍
- 수행 시간 👎

- 트리 균형 X
- 최대 손실 값(max delta loss)을 갖는 리프 노드 분할
- 깊고 비대칭적인 규칙 트리
=> 예측 오류 손실 최소화

LightBGM 하이퍼 파라미터

LightBGM 하이퍼 파라미터

➔ 리프 중심 트리 분할 방식에 맞는
하이퍼 파라미터 설정 필요
(ex. max_depth 매우 크게 가짐)

튜닝 방안

- 1. num_leaves 개수 중심
min_child_samples, max_depth 조정
➔ 트리 최대 깊이 조절
➔ 모델의 복잡도 조절
- 2. learning_rate 감소 n_estimators 증가
- 3. regulation 적용: reg_lambda, reg_alpha
- 4. 학습 데이터 피쳐 개수, 데이터 샘플링 레코드 개수 조절
: colsample_bytree, subsample

주요 파라미터	LightGBMClassifier 파라미터명	설명
num_iterations	n_estimators	- default=100 - 반복 수행하려는 트리 개수 지정
learning_rate	learning_rate	- default=0.1 - 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값 (0~1)
max_depth	max_depth	- default=-1 (제한X) - 트리 최대 깊이
min_data_in_leaf	min_child_samples	- default=20 - 최종 결정 클래스인 리프 노드가 되기 위해 필요한 최소 레코드 수
num_leaves	num_leaves	- default=31 - 하나의 트리가 가질 수 있는 최대 리프 개수
boosting	boosting	- default=gbdt : 일반적인 그래디언트 부스팅 결정 트리 - rf : 랜덤 포레스트 - 부스팅 트리 생성하는 알고리즘
bagging_fraction	subsample	- default=1.0 - 데이터 샘플링 비율
feature_fraction	colsample_bytree	- default=1.0 - 개별 트리를 학습할 때마다 무작위로 선택하는 피쳐의 비율
lambda_l2	reg_lambda	- default=1.0 - L2 regulation 제어. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있음
lambda_l1	reg_alpha	- default=0.0 - L1 regulation 제어
objective	objective	- 최솟값을 가져야 할 손실함수

LightGBM 적용

- 위스콘신 유방암 예측

- LightGBM 파이썬 패키지 lightgbm에서 **LGBMClassifier** 임포트
- 위스콘신 유방암 데이터 로드 -> 데이터 분리
- LGBMClassifier 객체 생성
 - n_estimators=400
 - learning_rate=0.05
 - *verbosity=-1 : 경고 로그 출력 방지*

```
from lightgbm import LGBMClassifier
```

```
# 데이터 로드
dataset = load_breast_cancer()

cancer_df = pd.DataFrame(data=dataset.data, columns=dataset.feature_names)
cancer_df['target']=dataset.target
X_features = cancer_df.iloc[:, :-1]
y_label = cancer_df.iloc[:, -1]

# 전체 데이터 -> 학습용 데이터 80%, 검증용 데이터 20%로 추출
X_train, X_test, y_train, y_test = train_test_split(X_features, y_label, test_size=0.2, random_state=156)

# X_train, y_train -> 학습용 데이터 90%, 검증용 데이터 10%로 분리
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=156)
```

LightBGM 적용

- 위스콘신 유방암 예측

- LightBGM으로 학습 데이터 학습
 - *callbacks*로 조기 중단: `stopping_rounds=50` => 111번 반복 후 학습 종료
 - 조기 중단 평가 지표 `eval_metric="logloss"`
 - 성능 평가 데이터 세트 `eval_set=evals`

```
# 조기 중단 수행 가능
# 버전 문제 - 콜백 내부에서 처리하는 방식 - 로 코드 수정
from lightgbm import early_stopping, log_evaluation

evals = [(X_tr, y_tr), (X_val, y_val)]
# verbose 파라미터 대신 log_evaluation으로 로그 출력 횟수 지정
lgbm_wrapper.fit(X_tr, y_tr, eval_metric="logloss", eval_set=evals,
                 callbacks=[early_stopping(stopping_rounds=50), log_evaluation(1)])
preds = lgbm_wrapper.predict(X_test)
pred_proba = lgbm_wrapper.predict_proba(X_test)[: , 1]
```

**** 버전 문제 -> 조기 중단 및 로그 출력 방식 수정**

```
[1] training's binary_logloss: 0.625671 valid_1's binary_logloss: 0.628248
Training until validation scores don't improve for 50 rounds
[2] training's binary_logloss: 0.588173 valid_1's binary_logloss: 0.601106
[3] training's binary_logloss: 0.554518 valid_1's binary_logloss: 0.577587
[4] training's binary_logloss: 0.522072 valid_1's binary_logloss: 0.556224
```

...

```
[106] training's binary_logloss: 0.0102381 valid_1's binary_logloss: 0.282058
[107] training's binary_logloss: 0.00986714 valid_1's binary_logloss: 0.279275
[108] training's binary_logloss: 0.00950998 valid_1's binary_logloss: 0.281427
[109] training's binary_logloss: 0.00915965 valid_1's binary_logloss: 0.280752
[110] training's binary_logloss: 0.00882581 valid_1's binary_logloss: 0.282152
[111] training's binary_logloss: 0.00850714 valid_1's binary_logloss: 0.280894
Early stopping, best iteration is:
[61] training's binary_logloss: 0.0532381 valid_1's binary_logloss: 0.260236
```

LightBGM 적용

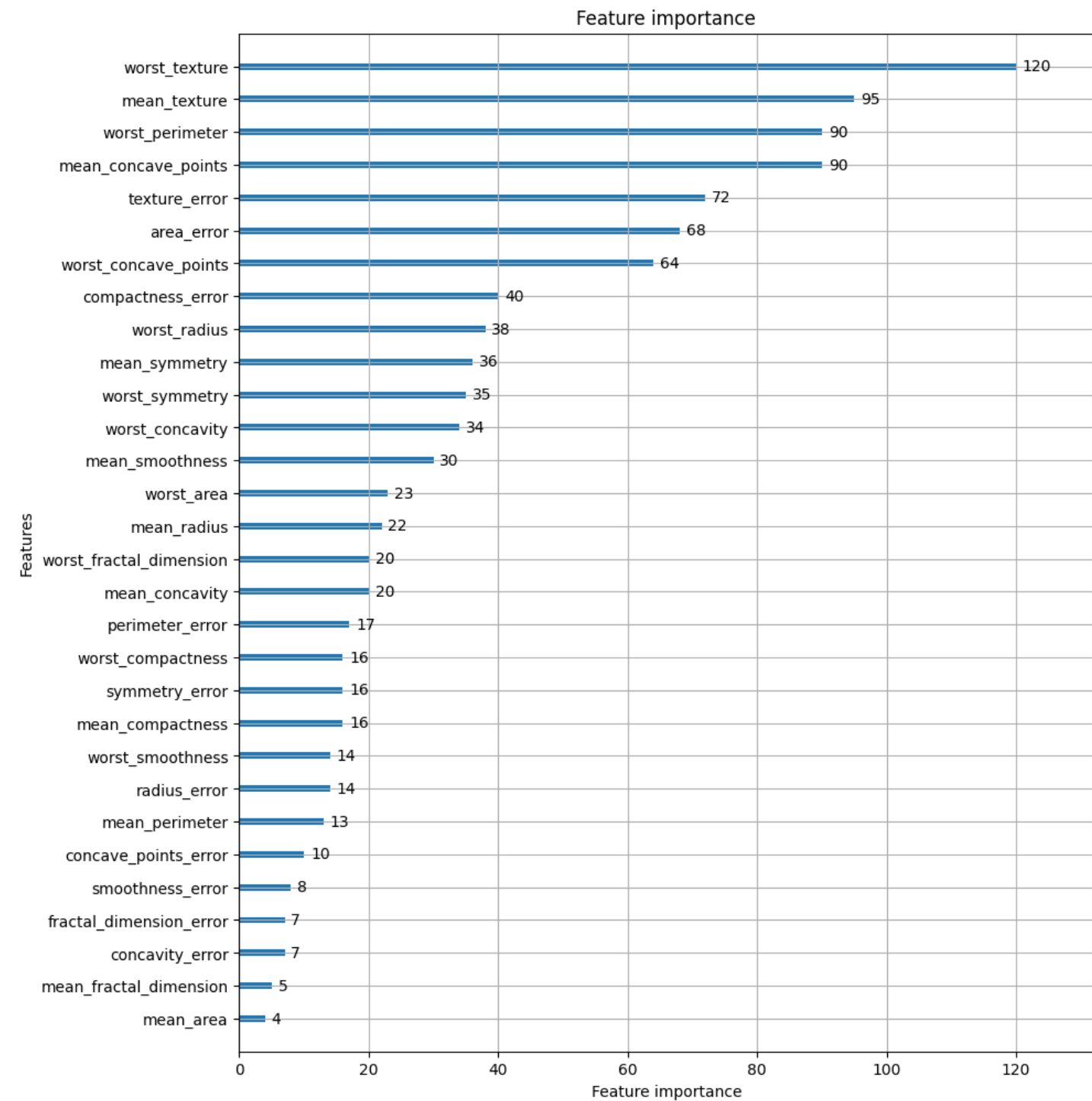
- 위스콘신 유방암 예측

- get_clf_eval() 예측 성능 평가

```
# 예측 성능 평가
get_clf_eval(y_test, preds, pred_proba)

오차 행렬
[[34  3]
 [ 2 75]]
정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740,      F1:0.9677, AUC:0.9877
```

- plot_importance(): lightgbm의 피쳐 중요도 시각화 API



4.8 베이지안 최적화



LightGBM 하이퍼 파라미터 튜닝하기

XGBoost, LightGBM: 하이퍼 파라미터 개수 ↑

-> GridSearch 방식: 하이퍼 파라미터 개수 많으면 최적화 수행 시간이 오래 걸림

```
params = {  
    'max_depth' = [10, 20, 30, 40, 50], 'num_leaves' = [ 35, 45, 55, 65],  
    'colsample_bytree'=[0.5, 0.6, 0.7, 0.8, 0.9], 'subsample' = [0.5, 0.6, 0.7, 0.8, 0.9],  
    'min_child_weight' = [10, 20, 30, 40], reg_alpha=[0.01, 0.05, 0.1]  
}
```

-> $5 \times 4 \times 5 \times 5 \times 4 \times 3 = 6000$ 회 반복 학습/평가 수행

베이지안 최적화

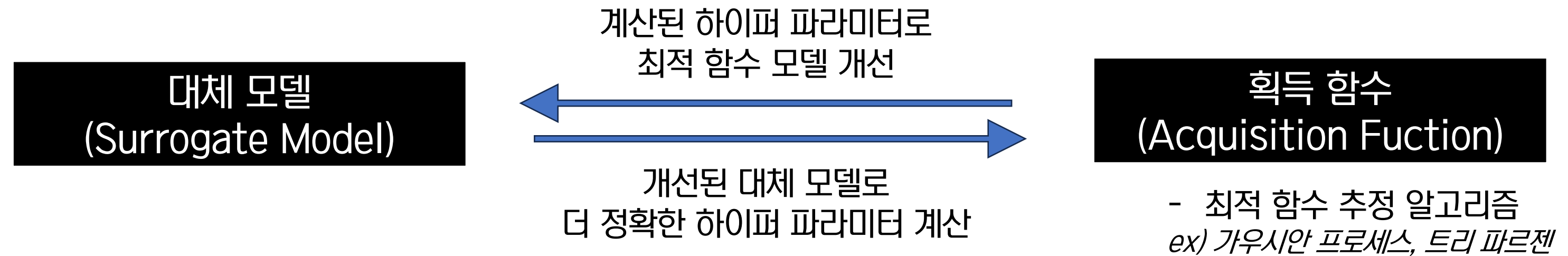
= 최대 또는 최소 함수 반환 값을 만드는 최적 입력 값을 빠르고 효과적으로 찾아주는 방식

= 새로운 데이터 입력 -> 최적 함수 예측 사후 모델 개선 => 최적 함수 모델 생성

$$f(x, y) = 2x - 3y \quad \rightarrow x=\text{최댓값}, y=0 \text{일 때 반환값이 최대}$$

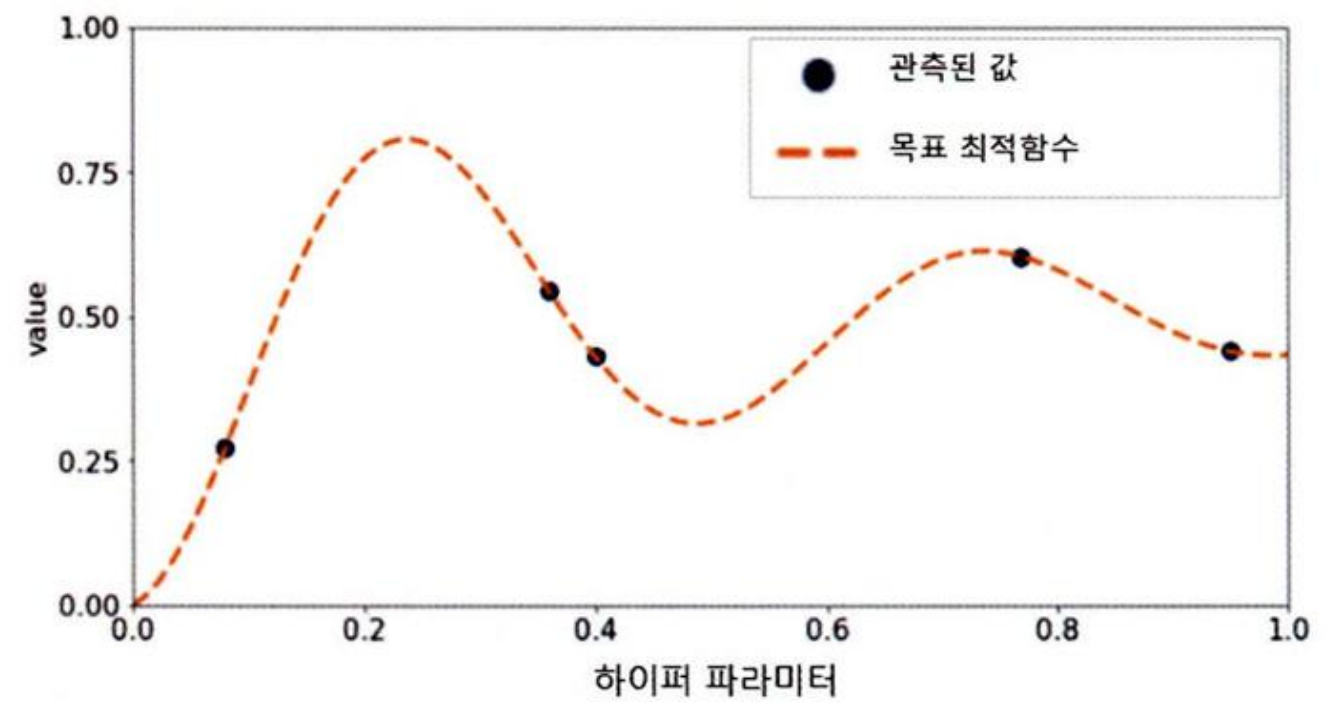
베이지안 최적화

베이지안 최적화의 구조



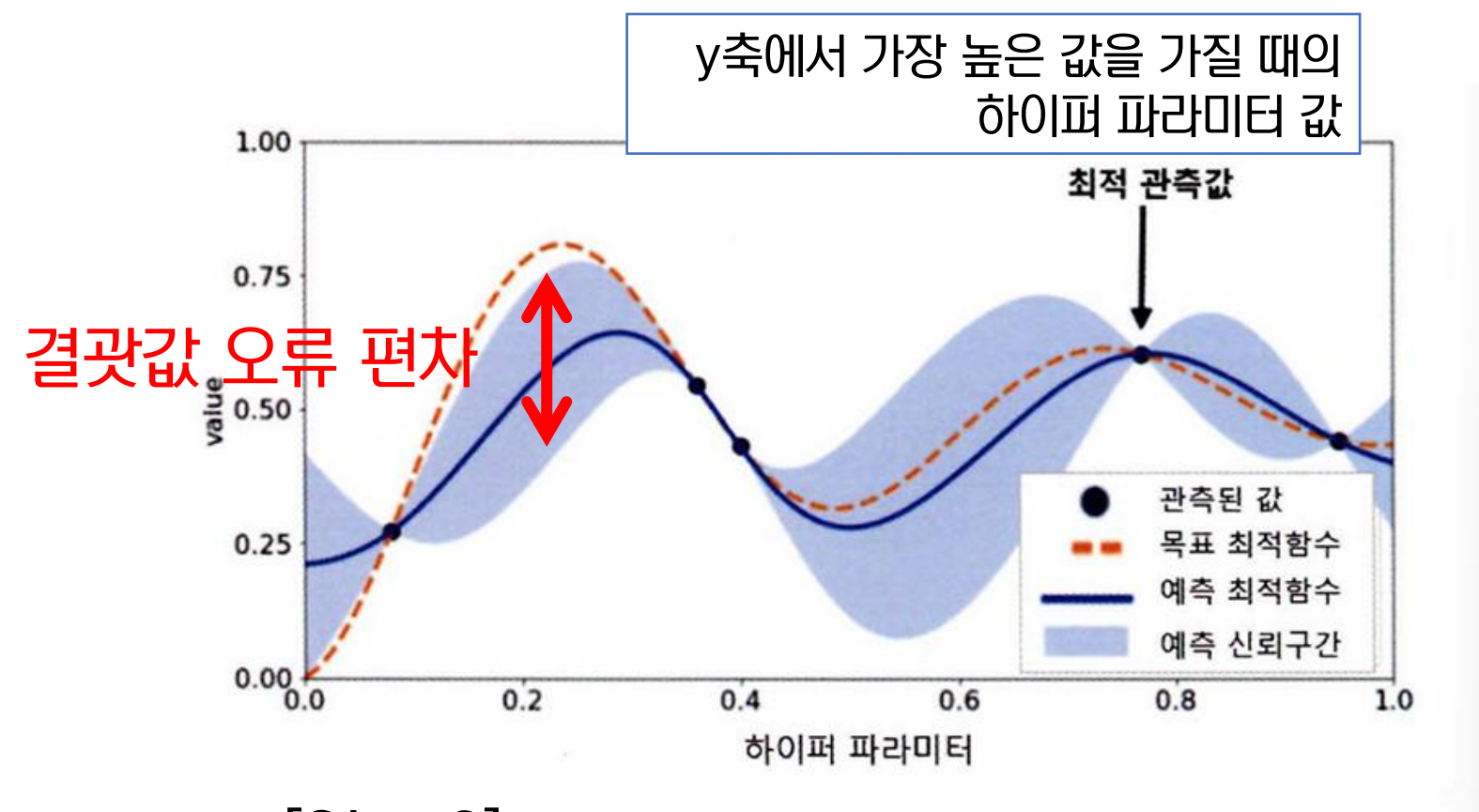
베이지안 최적화

베이지안 최적화 과정



[Step1]

랜덤 하이퍼 파라미터 샘플링, 성능 결과 관측

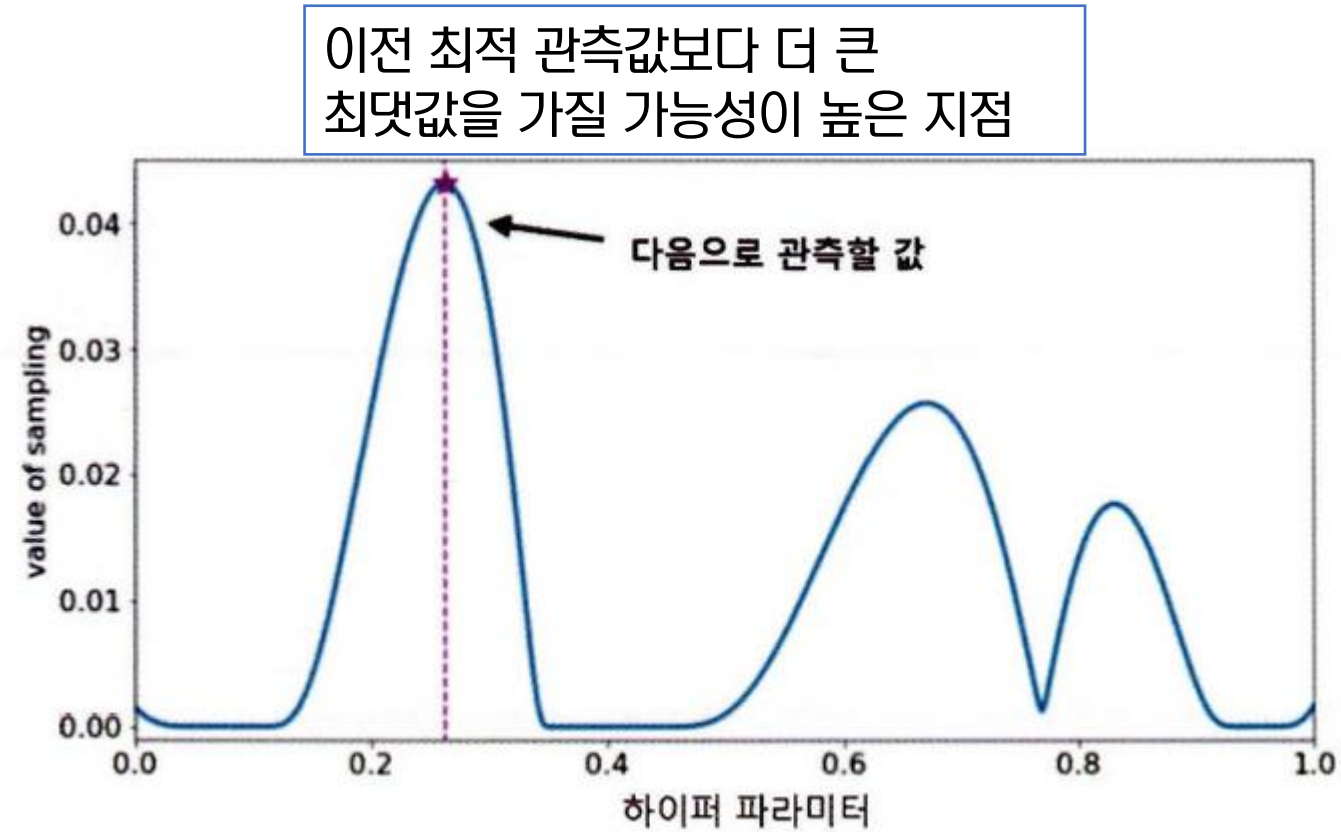


[Step2]

대체 모델: 관측된 값 기반 최적 함수 추정

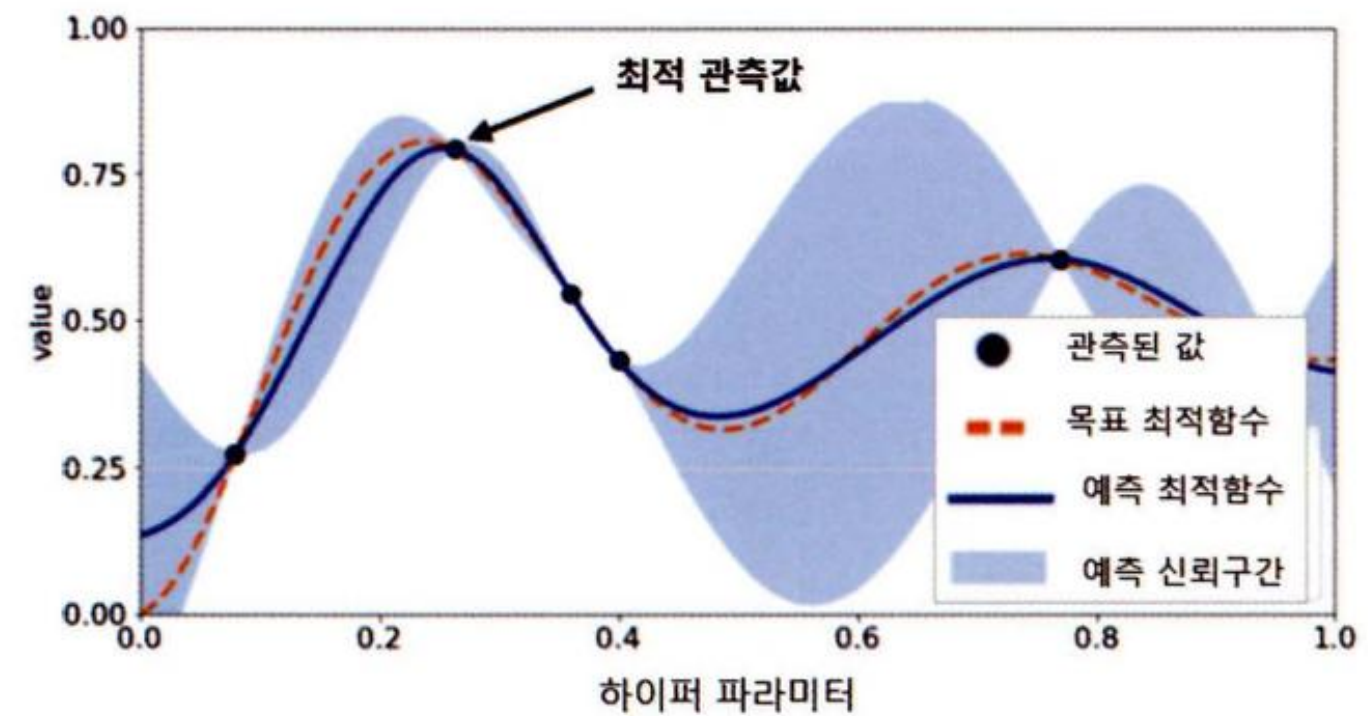
베이지안 최적화

베이지안 최적화 과정



[Step3]

획득 함수: 다음에 관측할 하이퍼 파라미터 값 계산
→ 대체 모델에 전달



[Step4]

획득 함수로부터 전달된 하이퍼 파라미터 수행
→ **대체 모델** 갱신
→ 최적 함수 다시 예측 및 추정

반복

→ 불확실성 개선, 정확한 최적 함수에 가까워짐

베이지안 최적화 – HyperOpt

베이지안 최적화 적용 파이썬 패키지

-> HyperOpt, Bayesian Optimization, Optuna 등

```
from hyperopt import hp
```

HyperOpt 활용 로직

1. 입력 변수명, 입력값의 **검색 공간** 설정
2. **목적 함수**(Objective Function) 설정
3. 목적 함수의 반환 **최솟값**을 가지는 최적 입력값 유추

베이지안 최적화 – HyperOpt

1. 입력 변수명, 입력값의 검색 공간 설정 -> 파이썬 딕셔너리 형태 { '변수명': (입력 변수의 검색 공간) }

```
### 입력 변수명과 입력값의 검색 공간 설정
```

```
# -10~10, 간격이 1인 입력 변수 x와 -15~15, 간격이 1인 입력 변수 y 설정
```

```
search_space = {'x': hp.quniform('x', -10, 10, 1), 'y': hp.quniform('y', -15, 15, 1)}
```

-> $x = [-10, -9, \dots, 9, 10]$, $y = [-15, -14, \dots, 14, 15]$

입력값 검색 공간 제공 함수

- **hp.quniform(label, low, high, q)**: 최솟값 low, 최댓값 high, 간격 q
- **hp.uniform(label, low, high)**: 최솟값 low, 최댓값 high, 정규 분포 형태 검색 공간
- **hp.randint(label, upper)**: 0~upper 중 랜덤한 정숫값으로 검색 공간 설정
- **hp.loguniform(label, low, high)**: $\exp(\text{uniform}(\text{low}, \text{high}))$ 값 반환
반환 값 log 변환 -> 정규 분포 형태 검색 공간
- **hp.choice(label, options)**: 검색 값에 문자열이 있을 경우 설정
입력 변수 값을 Options의 리스트, 튜플에 있는 문자열로 설정

베이지안 최적화 – HyperOpt

2. 목적 함수(Objective Function) 설정

- 파라미터: 입력값 검색 공간 딕셔너리
- 함숫값을 반환

```
### 목적 함수 생성

from hyperopt import STATUS_OK

# 목적 함수: 변수값과 변수 검색 공간을 가지는 딕셔너리를 인자로 받고, 특정 값을 반환
# retval = x**2 - 20*y로 계산된 값을 반환하는 목적 함수
def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y
    return retval
```

-> 설정한 x, y 입력값에 대한 함숫값 반환

목적 함수 $f(x, y) = x^2 - 20y$

베이지안 최적화 – HyperOpt

3. 목적 함수의 반환 최솟값을 가지는 최적 입력값 유추 – `fmin()`

`fmin()` 함수의 주요 파라미터

파라미터	설명
<code>fn</code>	- 목적 함수
<code>space</code>	- 공간 딕셔너리
<code>algo</code>	- 베이지안 최적화 적용 알고리즘 - default= <code>tpe.suggest</code> (= Tree of Parzen Estimator)
<code>max_evals</code>	- 입력값 입력 시도 횟수
<code>trials</code>	- 시도한 입력값 및 반환값 결과 저장 - Trials 객체 변수명으로 입력
<code>rstate</code>	- 랜덤 시드 값

ex) `max_evals=5`

```
### 목적 함수 반환값이 최소가 되는 최적의 하이퍼 파라미터 찾기 - 베이지안 최적화 기법
from hyperopt import fmin, tpe, Trials

# 입력 결과값을 저장한 Trials 객체값 생성
trial_val = Trials()

# 목적 함수의 최솟값을 반환하는 최적 입력 변수값을 5번의 입력값 시도로 알아내기
best_01 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=5,
               trials=trial_val, rstate=np.random.default_rng(seed=0))
print('best:', best_01)
```

100%|██████████| 5/5 [00:00<00:00, 537.18trial/s, best loss: -224.0]
best: {'x': np.float64(-4.0), 'y': np.float64(12.0)}

ex) `max_evals=20`

```
trial_val = Trials()

# max_evals=20으로 늘려서 재테스트
best_02 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=20,
               trials=trial_val, rstate=np.random.default_rng(seed=0))
print('best:', best_02)
```

100%|██████████| 20/20 [00:00<00:00, 792.37trial/s, best loss: -296.0]
best: {'x': np.float64(2.0), 'y': np.float64(15.0)}

베이지안 최적화 – HyperOpt

3. 목적 함수의 반환 최솟값을 가지는 최적 입력값 유추 – `fmin()`

`fmin()` 함수의 주요 파라미터

파라미터	설명
<code>fn</code>	- 목적 함수
<code>space</code>	- 공간 딕셔너리
<code>algo</code>	- 베이지안 최적화 적용 알고리즘 - default= <code>tpe.suggest</code> (= Tree of Parzen Estimator)
<code>max_evals</code>	- 입력값 입력 시도 횟수
<code>trials</code>	- 시도한 입력값 및 반환값 결과 저장 - Trials 객체 변수명으로 입력
<code>rstate</code>	- 랜덤 시드 값

-> Trials 객체의 `result` 속성
: 파이썬 리스트로 목적 함수의 반환값 저장

`vals` 속성
: 딕셔너리로 함수 수행마다 입력되는 입력값들 리스트 저장

베이지안 최적화 – HyperOpt

3. 목적 함수의 반환 최소값을 가지는 최적 입력값 유추 – `fmin()`

```
# Trials.results, Trials.vals를 DataFrame으로 만들어 확인하기
import pandas as pd

# results에서 loss 키값에 해당하는 value 추출 -> list 생성
losses = [loss_dict['loss'] for loss_dict in trial_val.results]

result_df = pd.DataFrame({'x': trial_val.vals['x'], 'y': trial_val.vals['y'], 'losses': losses})
result_df
```

입력 변수: x, y
반환값: loss

	x	y	losses
0	-6.0	5.0	-64.0
1	-4.0	10.0	-184.0
2	4.0	-2.0	56.0
3	-4.0	12.0	-224.0
4	9.0	1.0	61.0
5	2.0	15.0	-296.0
6	10.0	7.0	-40.0
7	-9.0	-10.0	281.0
8	-8.0	0.0	64.0
9	-0.0	-5.0	100.0
10	-0.0	-3.0	60.0

4.10 분류 실습



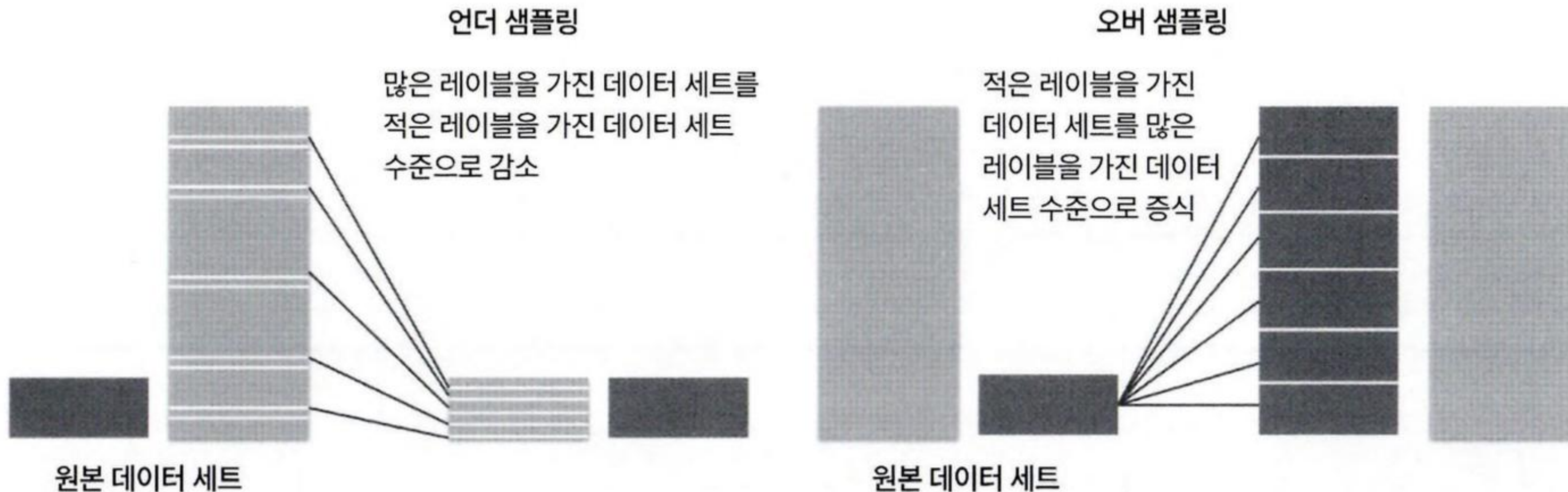
분류 실습 – 캐글 신용카드 사기 검출

- 신용카드 사기 검출 Class 레이블 분포
 - 0: 정상적인 신용카드 트랜잭션 데이터
 - 1: 신용카드 사기 트랜잭션
 - 극도로 불균형한 데이터 (사기 거래 비율: 0.172%)
 - 사기 검출(Fraud Detection), 이상 검출(Anomaly Detection) 데이터의 특징
 - 이상 거래 비율이 매우 적음
 - 대부분 정상 거래로 이루어짐
- > 불균형한 데이터 분포

언더 샘플링과 오버 샘플링의 이해

- 불균형한 데이터 분포를 가진 데이터 세트 학습의 문제점
 - 예측 성능 저하

-> 언더 샘플링과 오버 샘플링 활용



오버 샘플링 - SMOTE

- SMOTE(Synthetic Minority Over-sampling Technique)
- K 최근접 이웃(KNN) 기반 데이터 생성
- 기존 데이터와 유사한, 새로운 데이터 샘플 생성



데이터 일차 가공 및 모델 학습/예측/평가

- 아나콘다를 이용해 imbalabced-learn 패키지 설치

```
conda install -c conda-forge imbalanced-learn
```

- 새로운 주피터 노트북 생성 후, creditcard.csv 파일을 DataFrame으로 로딩

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline

card_df = pd.read_csv('./creditcard.csv')
card_df.head(3)
```

【Output】

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679

3 rows × 31 columns

데이터 일차 가공 및 모델 학습/예측/평가

- 불필요한 Time 피쳐 삭제

```
from sklearn.model_selection import train_test_split

# 인자로 입력받은 DataFrame을 복사한 뒤 Time 칼럼만 삭제하고 복사된 DataFrame 반환
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    df_copy.drop('Time', axis=1, inplace=True)
    return df_copy
```


데이터 일차 가공 및 모델 학습/예측/평가

- 학습 피처/레이블 데이터 세트, 테스트 피처/레이블 데이터 세트를 반환

```
# 사전 데이터 가공 후 학습과 테스트 데이터 세트를 반환하는 함수.
def get_train_test_dataset(df=None):
    # 인자로 입력된 DataFrame의 사전 데이터 가공이 완료된 복사 DataFrame 반환
    df_copy = get_preprocessed_df(df)
    # DataFrame의 맨 마지막 칼럼이 레이블, 나머지는 피처들
    X_features = df_copy.iloc[:, :-1]
    y_target = df_copy.iloc[:, -1]
    # train_test_split( )으로 학습과 테스트 데이터 분할. stratify=y_target으로 Stratified 기반 분할
    X_train, X_test, y_train, y_test = \
        train_test_split(X_features, y_target, test_size=0.3, random_state=0, stratify=y_target)
    # 학습과 테스트 데이터 세트 반환
    return X_train, X_test, y_train, y_test
```

```
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
```


데이터 일차 가공 및 모델 학습/예측/평가

- 학습 데이터 세트와 테스트 데이터 세트의 레이블 값 비율을 백분율로 환산

```
print('학습 데이터 레이블 값 비율')  
print(y_train.value_counts()/y_train.shape[0] * 100)  
print('테스트 데이터 레이블 값 비율')  
print(y_test.value_counts()/y_test.shape[0] * 100)
```

【Output】

```
학습 데이터 레이블 값 비율  
0    99.827451  
1     0.172549  
테스트 데이터 레이블 값 비율  
0    99.826785  
1     0.173215
```

데이터 일차 가공 및 모델 학습/예측/평가

- 로지스틱 회귀를 이용해 신용 카드 사기 여부를 예측

```
from sklearn.linear_model import LogisticRegression

lr_clf = LogisticRegression(max_iter=1000)
lr_clf.fit(X_train, y_train)
lr_pred = lr_clf.predict(X_test)
lr_pred_proba = lr_clf.predict_proba(X_test)[:, 1]

# 3장에서 사용한 get_clf_eval() 함수를 이용해 평가 수행.
get_clf_eval(y_test, lr_pred, lr_pred_proba)
```

【Output】

오차 행렬

```
[[85281   14]
 [   56   92]]
```

정확도: 0.9992, 정밀도: 0.8679, 재현율: 0.6216, F1: 0.7244, AUC:0.9702

데이터 일차 가공 및 모델 학습/예측/평가

- 반복적으로 모델을 변경해 학습/예측/평가 하기 위한 함수를 생성

```
# 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가 수행.  
def get_model_train_eval(model, ftr_train=None, ftr_test=None, tgt_train=None, tgt_test=None):  
    model.fit(ftr_train, tgt_train)  
    pred = model.predict(ftr_test)  
    pred_proba = model.predict_proba(ftr_test)[:, 1]  
    get_clf_eval(tgt_test, pred, pred_proba)
```


데이터 일차 가공 및 모델 학습/예측/평가

- LightGBM으로 모델을 학습 후, 예측 평가를 수행

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1, boost_from_average=False)
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train, tgt_test=y_test)
```

【Output】

오차 행렬

```
[[85290    5]
 [   36  112]]
```

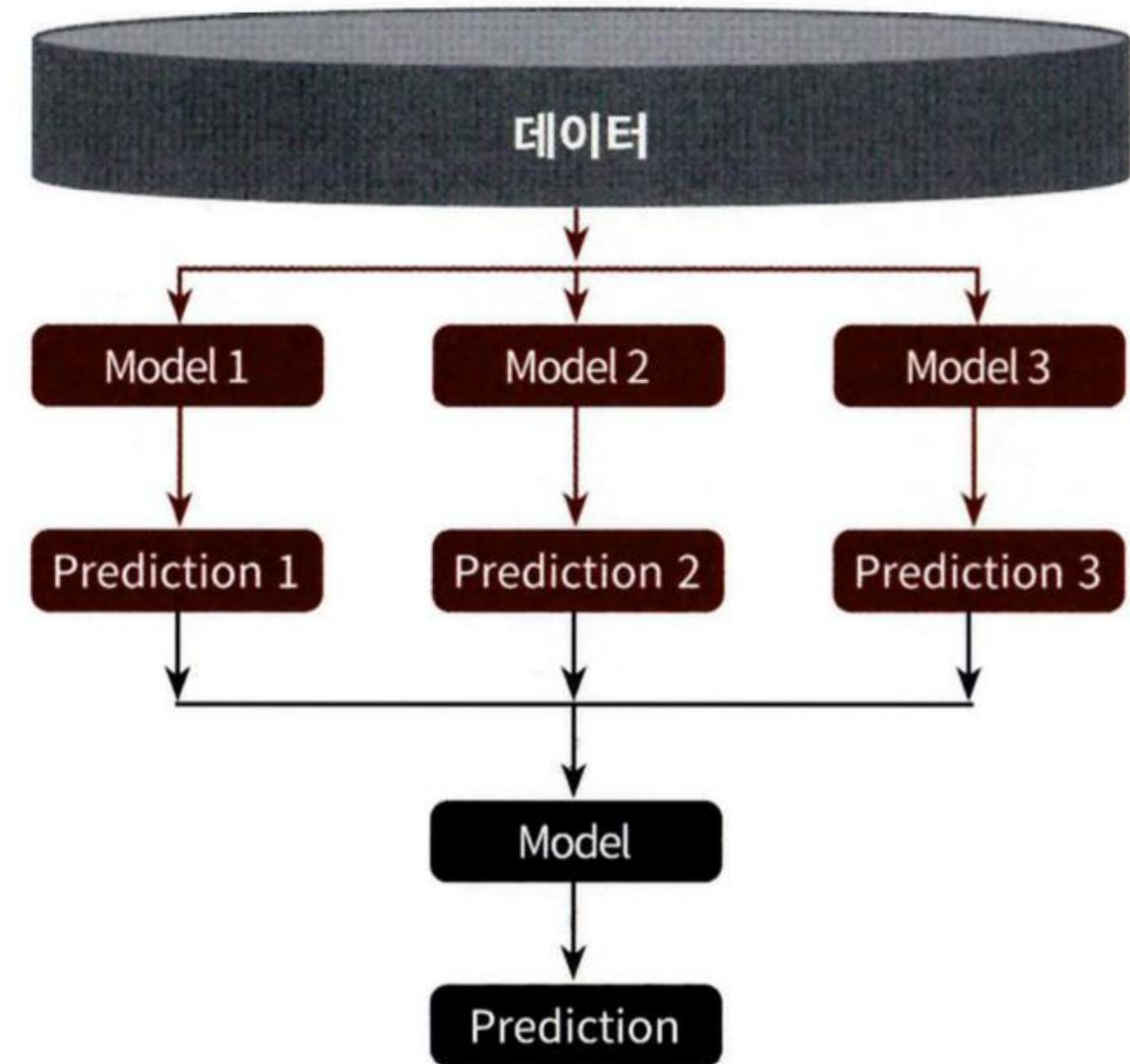
정확도: 0.9995, 정밀도: 0.9573, 재현율: 0.7568, F1: 0.8453, AUC:0.9790

4.11 스테킹 앙상블

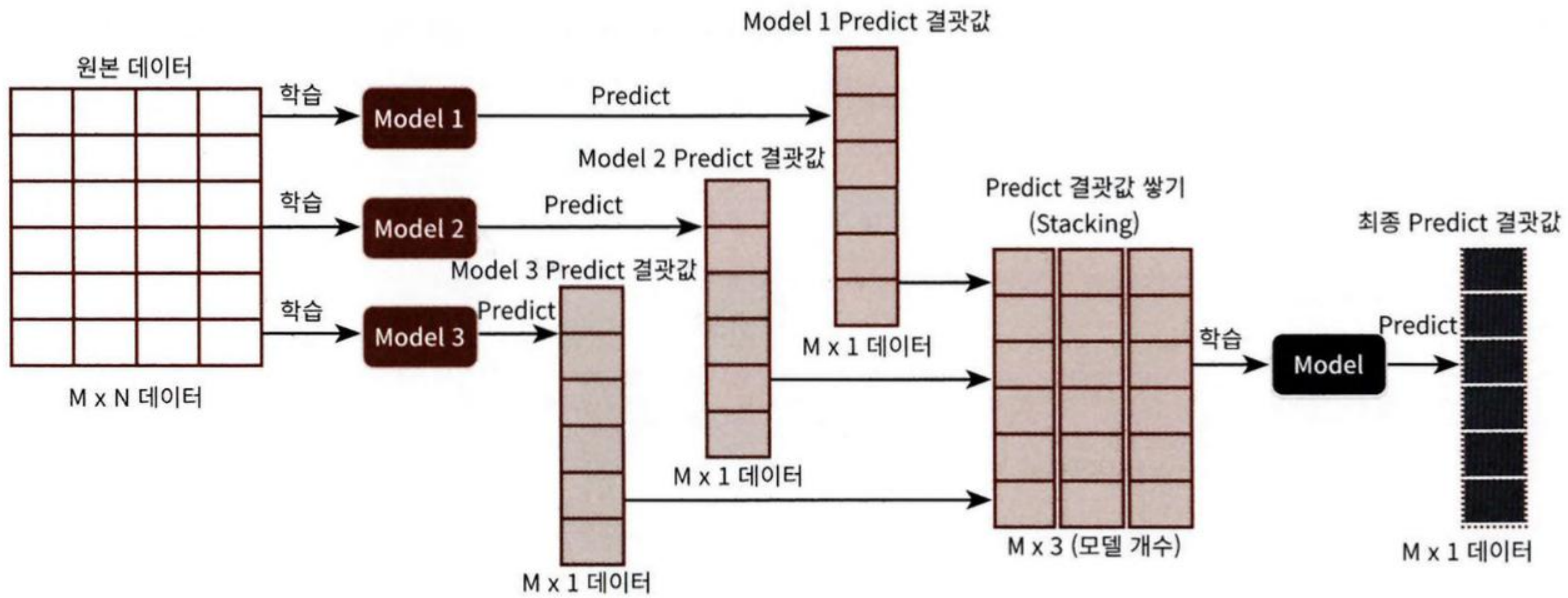


스택킹(Stacking) 앙상블

- 스택킹(Stacking) 개요
 - 여러 알고리즘을 결합해 예측 결과를 도출(배깅 & 부스팅과의 공통점)
 - 개별 알고리즘의 예측 결과 데이터를 기반으로 다시 예측을 수행
- 스택킹 모델이 필요한 두 종류의 모델
 - 개별적인 기반 모델
 - 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어, 학습하는 최종 메타 모델



스택킹(Stacking) 앙상블



스택킹(Stacking) 앙상블

- 예제: 기본 스택킹 모델

```
import numpy as np

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer_data = load_breast_cancer()

X_data = cancer_data.data
y_label = cancer_data.target

X_train, X_test, y_train, y_test = train_test_split(X_data, y_label, test_size=0.2, random_state=0)
```


스택킹(Stacking) 앙상블

- 스택킹에 사용될 머신러닝 알고리즘 클래스 생성
 - 개별모델: KNN, 랜덤 포레스트, 결정 트리, 에이다부스트
 - 최종 모델: 로지스틱 회귀

```
# 개별 ML 모델 생성
knn_clf = KNeighborsClassifier(n_neighbors=4)
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0)
dt_clf = DecisionTreeClassifier()
ada_clf = AdaBoostClassifier(n_estimators=100)

# 스택킹으로 만들어진 데이터 세트를 학습, 예측할 최종 모델
lr_final = LogisticRegression()
```

```
# 개별 모델들을 학습.
knn_clf.fit(X_train, y_train)
rf_clf.fit(X_train, y_train)
dt_clf.fit(X_train, y_train)
ada_clf.fit(X_train, y_train)
```

```
# 학습된 개별 모델들이 각자 반환하는 예측 데이터 세트를 생성하고 개별 모델의 정확도 측정.
knn_pred = knn_clf.predict(X_test)
rf_pred = rf_clf.predict(X_test)
dt_pred = dt_clf.predict(X_test)
ada_pred = ada_clf.predict(X_test)
print('KNN 정확도: {0:.4f}'.format(accuracy_score(y_test, knn_pred)))
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy_score(y_test, rf_pred)))
print('결정 트리 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))
print('에이다부스트 정확도: {0:.4f}'.format(accuracy_score(y_test, ada_pred)))
```

[Output]

```
KNN 정확도: 0.9211
랜덤 포레스트 정확도: 0.9649
결정 트리 정확도: 0.9123
에이다부스트 정확도: 0.9561
```


스택킹(Stacking) 앙상블

- 개별 알고리즘으로 예측된 값을 최종 메타 모델인 로지스틱 회귀에서 학습데이터로 사용

```
pred = np.array([knn_pred, rf_pred, dt_pred, ada_pred])  
print(pred.shape)  
  
# transpose를 이용해 행과 열의 위치 교환. 칼럼 레벨로 각 알고리즘의 예측 결과를 피처로 만듦.  
pred = np.transpose(pred)  
print(pred.shape)
```

【Output】

```
(4, 114)  
(114, 4)
```

스택킹(Stacking) 앙상블

- 로지스틱 회귀 예측 정확도 측정

```
lr_final.fit(pred, y_test)
final = lr_final.predict(pred)

print('최종 메타 모델의 예측 정확도: {0:.4f}'.format(accuracy_score(y_test, final)))
```

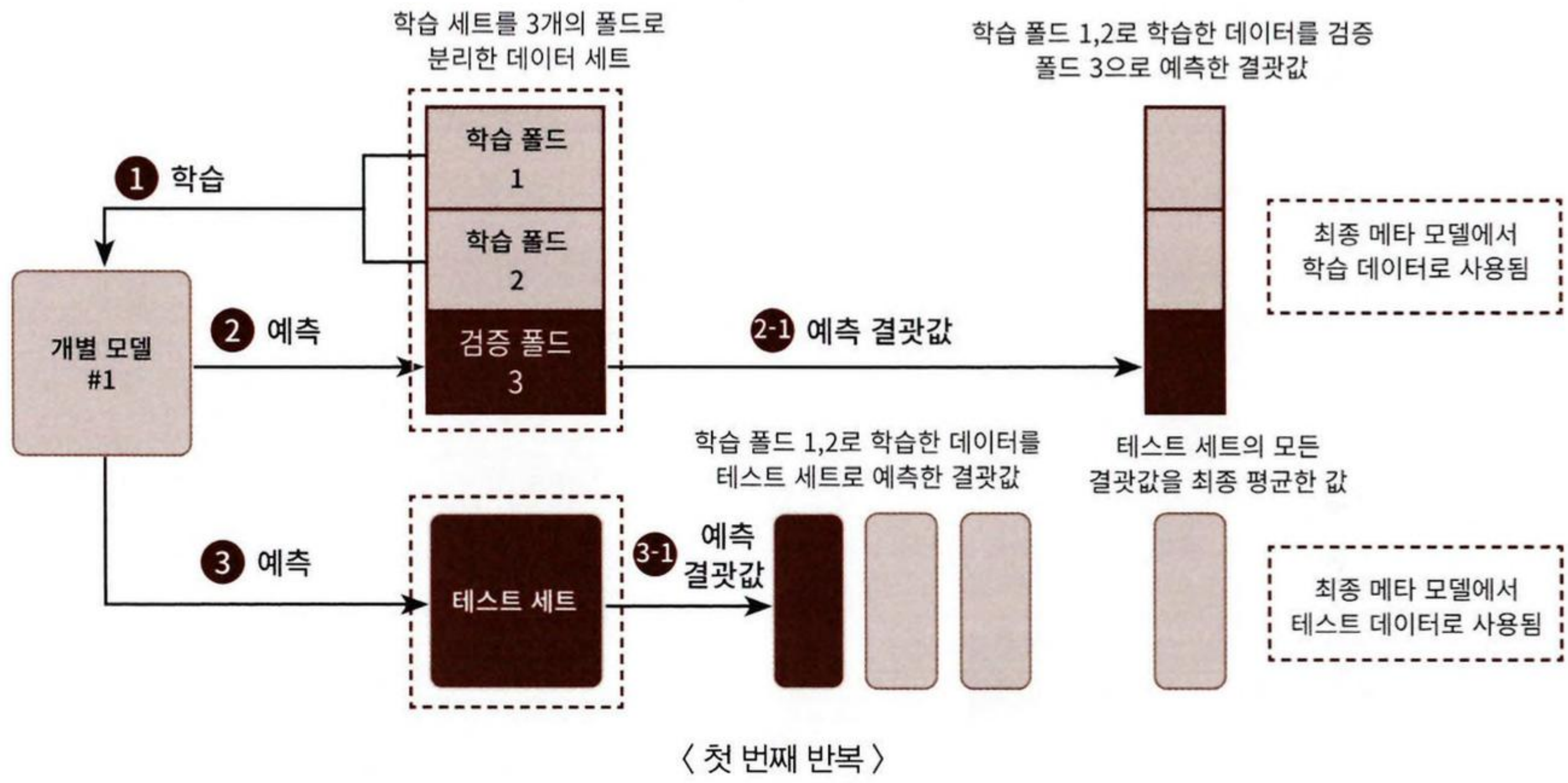
【Output】

최종 메타 모델의 예측 정확도: 0.9737

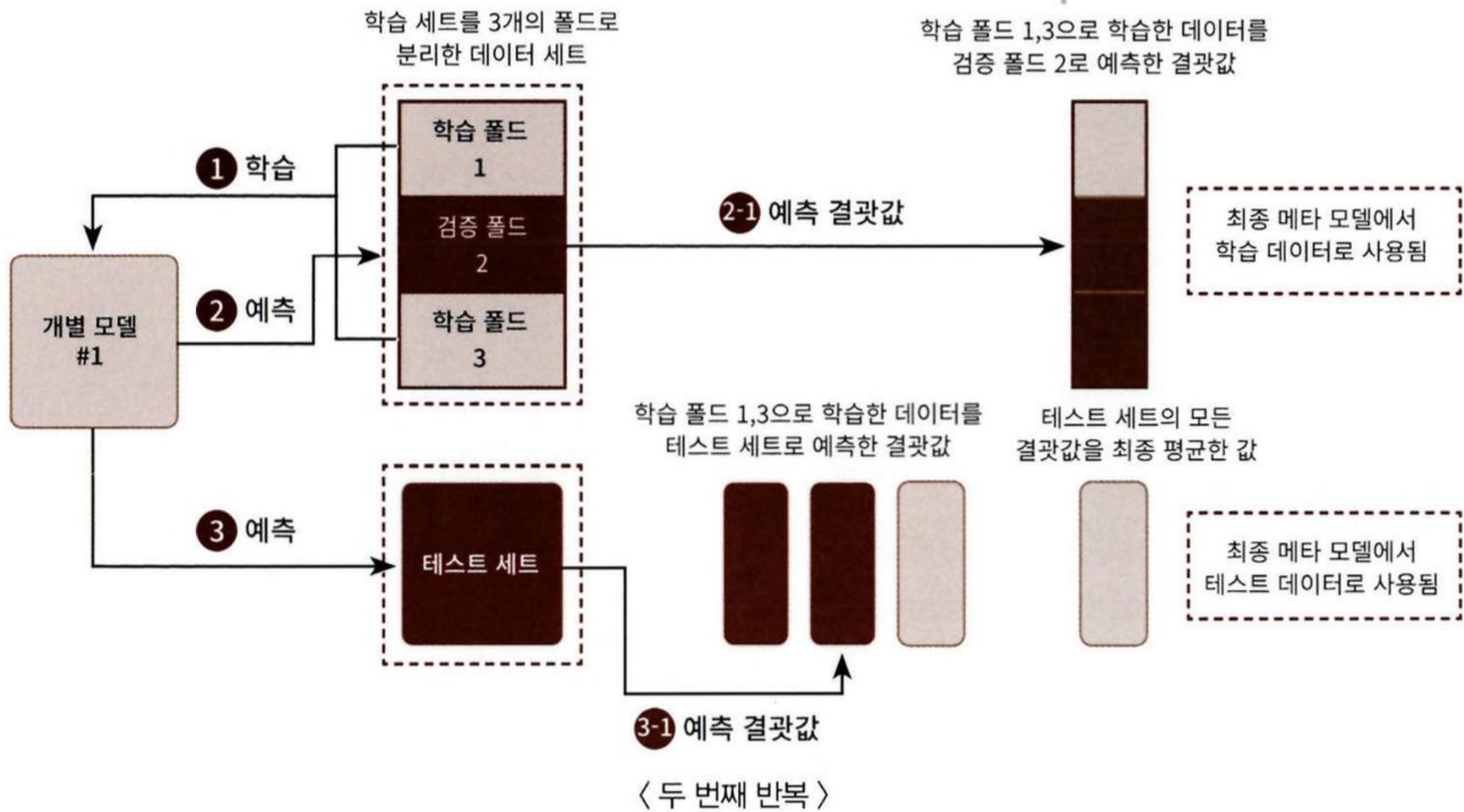
CV 세트 기반 스택킹

- 과적합 방지를 위해 Cross-Validation을 활용한 스택킹
- 2단계 스텝으로 구분
 - 스텝 1: 각 모델 별 원본 학습/테스트 데이터를 예측한 결과값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터 세트 생성
 - 스텝 2: 개별 모델이 생성한 학습용 데이터를 스택킹 형태로 합해, 메타 모델이 학습할 최종 학습용 데이터 세트 생성

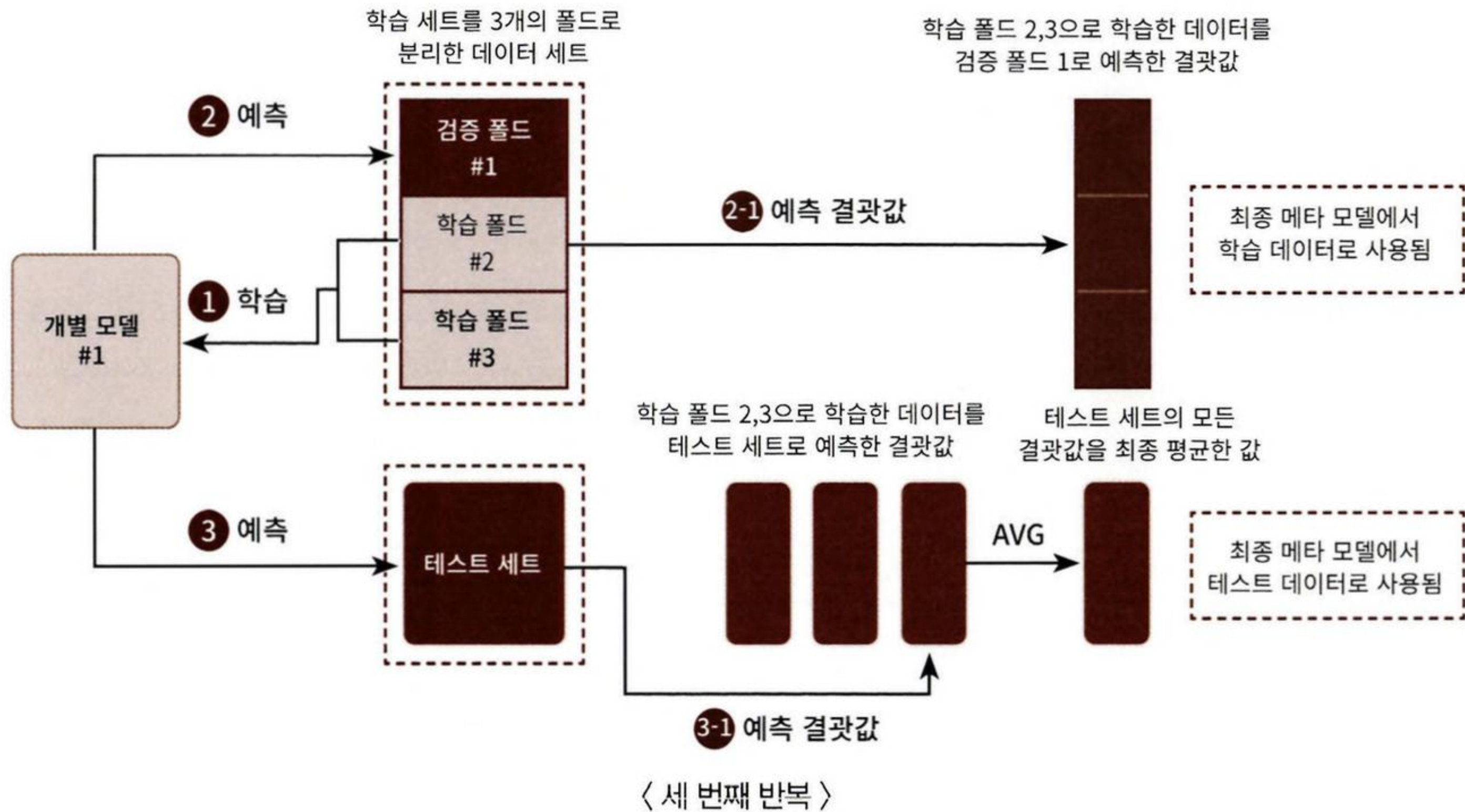
CV 세트 기반 스태킹



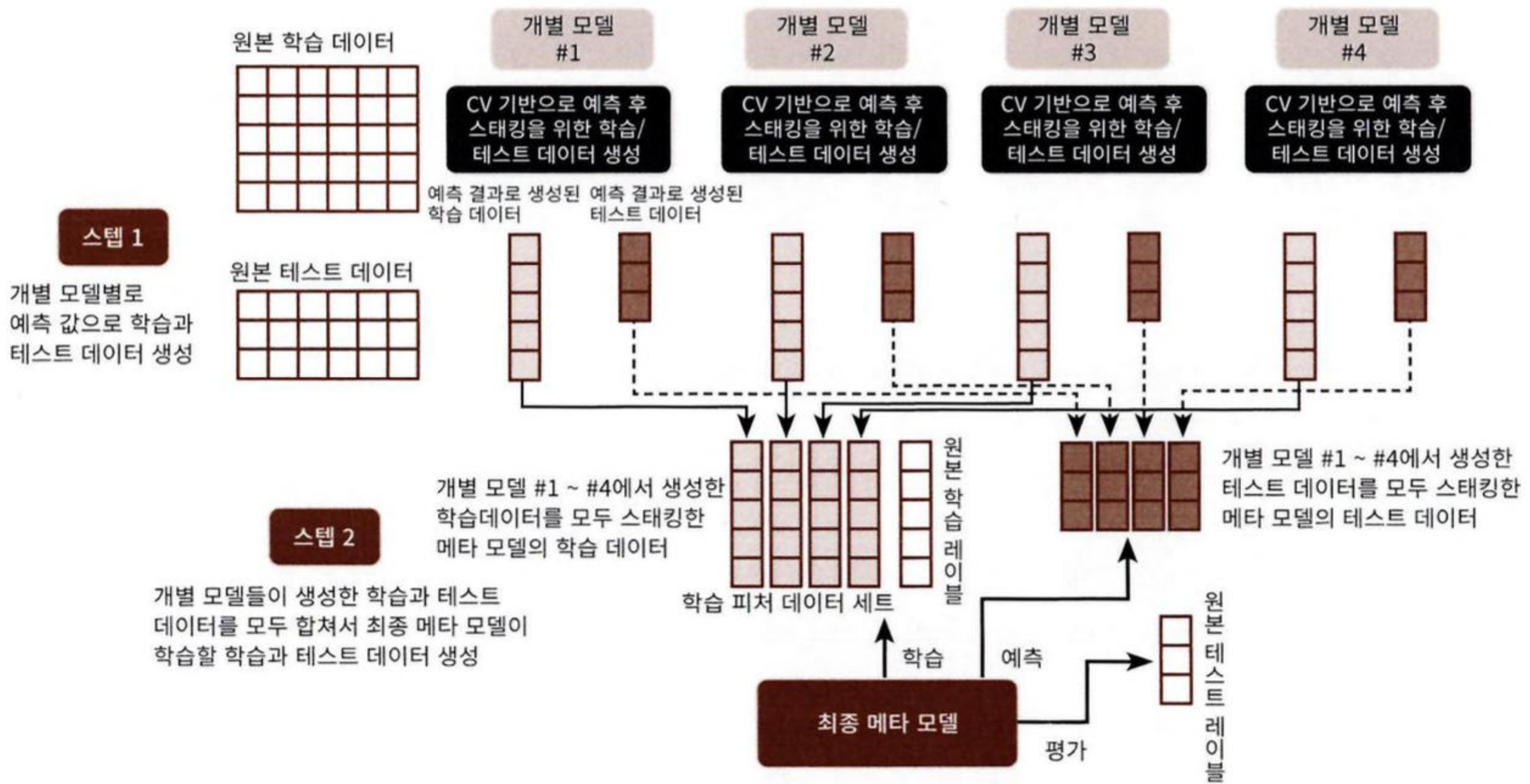
CV 세트 기반 스태킹



CV 세트 기반 스타킹



CV 세트 기반 스택킹



스택킹 앙상블

- 개별 모델의 Classifier 객체, 원본 학습 용 피쳐 데이터, 원본 학습용 레이블 데이터, 원본 테스트 피쳐 데이터, K폴드의 개수를 파라미터로 입력
- 예측 결과값을 기반으로 메타 모델을 위한 학습용 데이터와 테스트용 데이터를 새롭게 생성

```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error
```

```
# 개별 기반 모델에서 최종 메타 모델이 사용할 학습 및 테스트용 데이터를 생성하기 위한 함수.
def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds ):
    # 지정된 n_folds값으로 KFold 생성.
    kf = KFold(n_splits=n_folds, shuffle=False)
    # 추후에 메타 모델이 사용할 학습 데이터 반환을 위한 넘파이 배열 초기화
    train_fold_pred = np.zeros((X_train_n.shape[0], 1 ))
    test_pred = np.zeros((X_test_n.shape[0], n_folds))
    print(model.__class__.__name__, ' model 시작 ')

    for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):
```

```
# 입력된 학습 데이터에서 기반 모델이 학습/예측할 폴드 데이터 세트 추출
print('\t 폴드 세트: ', folder_counter, ' 시작 ')
X_tr = X_train_n[train_index]
y_tr = y_train_n[train_index]
X_te = X_train_n[valid_index]
```

```
# 폴드 세트 내부에서 다시 만들어진 학습 데이터로 기반 모델의 학습 수행.
model.fit(X_tr, y_tr)
# 폴드 세트 내부에서 다시 만들어진 검증 데이터로 기반 모델 예측 후 데이터 저장.
train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1, 1)
# 입력된 원본 테스트 데이터를 폴드 세트내 학습된 기반 모델에서 예측 후 데이터 저장.
test_pred[:, folder_counter] = model.predict(X_test_n)
```

```
# 폴드 세트 내에서 원본 테스트 데이터를 예측한 데이터를 평균하여 테스트 데이터로 생성
test_pred_mean = np.mean(test_pred, axis=1).reshape(-1, 1)
```

```
#train_fold_pred는 최종 메타 모델이 사용하는 학습 데이터, test_pred_mean은 테스트 데이터
return train_fold_pred, test_pred_mean
```


스택킹 앙상블

- 메타 모델이 추후에 사용할 학습용, 테스트용 데이터 세트를 반환

```
knn_train, knn_test = get_stacking_base_datasets(knn_clf, X_train, y_train, X_test, 7)
rf_train, rf_test = get_stacking_base_datasets(rf_clf, X_train, y_train, X_test, 7)
dt_train, dt_test = get_stacking_base_datasets(dt_clf, X_train, y_train, X_test, 7)
ada_train, ada_test = get_stacking_base_datasets(ada_clf, X_train, y_train, X_test, 7)
```

스택킹 앙상블

- 반환된 각 모델 별 학습 데이터와 테스트 데이터를 합함

```
Stack_final_X_train = np.concatenate((knn_train, rf_train, dt_train, ada_train), axis=1)
Stack_final_X_test = np.concatenate((knn_test, rf_test, dt_test, ada_test), axis=1)
print('원본 학습 피쳐 데이터 Shape:', X_train.shape, '원본 테스트 피쳐 Shape:', X_test.shape)
print('스택킹 학습 피쳐 데이터 Shape:', Stack_final_X_train.shape,
      '스택킹 테스트 피쳐 데이터 Shape:', Stack_final_X_test.shape)
```

【Output】

```
원본 학습 피쳐 데이터 Shape: (455, 30) 원본 테스트 피쳐 Shape: (114, 30)
스택킹 학습 피쳐 데이터 Shape: (455, 4) 스택킹 테스트 피쳐 데이터 Shape: (114, 4)
```


스택킹 앙상블

- 로지스틱 회귀를 스택킹된 학습용 피쳐 데이터 세트와 원본 학습 레이블 데이터로 학습
- 예측 결과를 원본 테스트 레이블 데이터와 비교하여 정확도를 측정

```
lr_final.fit(Stack_final_X_train, y_train)
stack_final = lr_final.predict(Stack_final_X_test)

print('최종 메타 모델의 예측 정확도: {0:.4f}'.format(accuracy_score(y_test, stack_final)))
```

【Output】

```
최종 메타 모델의 예측 정확도: 0.9825
```

THANK YOU

